

目录

1	ESP-Claw 项目架构文档	2
1.1	目录	2
1.2	1. 项目概述	3
1.2.1	1.2.1 核心理念	3
1.2.2	1.2.2 技术栈	3
1.2.3	1.2.3 支持的硬件	3
1.2.4	1.2.4 技术概念讲解	4
1.3	2. 总体架构分层	4
1.3.1	1.3.1 数据流概览	5
1.3.2	1.3.2 目录结构总览	5
1.3.3	1.3.3 技术概念讲解	6
1.4	3. 启动流程详解	6
1.4.1	1.4.1 3.1 启动序列	6
1.4.2	1.4.2 3.2 app_claw_start() 详细初始化	8
1.4.3	1.4.3 3.3 FreeRTOS 任务全景图	9
1.4.4	1.4.4 技术概念讲解	10
1.5	4. 核心 Agent 引擎	11
1.5.1	1.5.1 4.1 claw_core —Agent 推理循环	11
1.5.2	1.5.2 技术概念讲解	14
1.5.3	1.5.3 4.2 claw_event_router —事件路由器	16
1.5.4	1.5.4 技术概念讲解	18
1.5.5	1.5.5 4.3 claw_cap —能力插件系统	18
1.5.6	1.5.6 技术概念讲解	20
1.5.7	1.5.7 4.4 claw_memory —记忆系统	20
1.5.8	1.5.8 技术概念讲解	22
1.5.9	1.5.9 4.5 claw_skill —技能系统	23
1.5.10	1.5.10 技术概念讲解	24
1.6	5. 能力层详解	25
1.6.1	1.6.1 默认对 LLM 可见的能力组 (llm_visible_by_default = true)	25
1.6.2	1.6.2 默认对 LLM 不可见的能力组 (llm_visible_by_default = false)	25
1.6.3	1.6.3 技术概念讲解	28
1.7	6. 通信与网络层	30
1.7.1	1.7.1 6.1 Wi-Fi 管理器	30
1.7.2	1.7.2 6.2 HTTP 服务器	31
1.7.3	1.7.3 6.3 Captive DNS 配网门户	31
1.7.4	1.7.4 6.4 HTTP 连接复用	31
1.7.5	1.7.5 6.5 MCP 协议 (Model Context Protocol)	31
1.7.6	1.7.6 6.6 IM 平台集成	32
1.7.7	1.7.7 技术概念讲解	32
1.8	7. Lua 脚本系统	34
1.8.1	1.8.1 7.1 架构模型	34
1.8.2	1.8.2 7.2 模块分类	34
1.8.3	1.8.3 7.3 LLM 动态编程	35
1.8.4	1.8.4 7.4 模块安全锁	36
1.8.5	1.8.5 技术概念讲解	36
1.9	8. 硬件与显示系统	38

1.9.1	8.1 Board Manager	38
1.9.2	8.2 情绪动画系统 (Hiyori)	38
1.9.3	8.3 视频播放引擎	38
1.9.4	8.4 表情引擎 (Emote)	38
1.9.5	8.5 显示仲裁器 (Display Arbiter)	39
1.9.6	8.6 ESP Painter	39
1.9.7	8.7 音频系统	39
1.9.8	技术概念讲解	39
1.10	9. 存储架构	41
1.10.1	9.1 Flash 分区布局	41
1.10.2	9.2 FATFS 文件系统布局	42
1.10.3	9.3 NVS 配置存储	42
1.10.4	9.4 OTA 双槽位	43
1.10.5	技术概念讲解	43
1.11	10. 构建系统	45
1.11.1	10.1 构建流程	45
1.11.2	10.2 FATFS 镜像生成	46
1.11.3	10.3 ESP-IDF 补丁	46
1.11.4	10.4 外部依赖	46
1.11.5	10.5 CI/CD	47
1.11.6	技术概念讲解	47
1.12	附录: 关键文件索引	49
1.12.1	入口与配置	49
1.12.2	核心引擎	49
1.12.3	LLM 后端	49
1.12.4	网络	50
1.12.5	硬件与显示	50
1.12.6	构建	50

1 ESP-Claw 项目架构文档

1.1 目录

1. 项目概述
2. 总体架构分层
3. 启动流程详解
4. 核心 Agent 引擎
 - 4.1 claw_core —Agent 推理循环
 - 4.2 claw_event_router —事件路由器
 - 4.3 claw_cap —能力插件系统
 - 4.4 claw_memory —记忆系统
 - 4.5 claw_skill —技能系统
5. 能力层详解
6. 通信与网络层
7. Lua 脚本系统
8. 硬件与显示系统
9. 存储架构
10. 构建系统

1.2 1. 项目概述

ESP-Claw 是 Espressif Systems 开源的 AI Agent 物联网设备框架，采用 Apache 2.0 许可证。它让设备通过自然语言对话来定义行为（“Chat Coding”），并在 ESP32 系列芯片上**本地完成**感知 → 决策 → 执行的完整闭环。

1.2.1 核心理念

- **Chat as Creation (对话即编程)**：通过 IM 聊天界面与设备对话，动态加载 Lua 脚本，无需传统编程即可定义设备行为
- **Event Driven (事件驱动)**：任何事件都可以触发 Agent 推理循环，响应时间可达毫秒级
- **Structured Memory (结构化记忆)**：本地组织记忆数据，隐私不外泄
- **MCP Communication (MCP 通信)**：同时支持 MCP Server 和 Client，可接入标准 MCP 工具生态

1.2.2 技术栈

层级	技术
操作系统	FreeRTOS (ESP-IDF v5.5.4)
脚本引擎	Lua 5.5
图形框架	LVGL v9.4.0
网络协议	Wi-Fi 802.11 b/g/n, HTTP/HTTPS, WebSocket, mDNS
LLM 后端	OpenAI, Anthropic, 阿里云百炼 (Qwen), DeepSeek, 自定义端点
IM 平台	Telegram, QQ, 飞书, 微信, WebIM
音频	Opus 编解码, PCM, I2S 音频输出, 语音识别 (ASR)
构建系统	CMake + ESP-IDF 组件管理

1.2.3 支持的硬件

项目支持 **17 种板卡配置**，涵盖 7 个厂商，主要芯片平台为 ESP32-P4 和 ESP32-S3：

厂商	板卡
Espressif	ESP32-P4-Eye, P4-Function-EV, P4-WiFi6-Touch-LCD-4.3, S3-DevKitC-1, ESP-Box-3, SensairShuttle, Vocat-Board
M5Stack	CoreS3, StickS3
DFRobot	K10
LilyGO	T-Display-S3
Movell	Cuican-ESP32S3, Moji-ESP32S3, Moji2-ESP32C5
Waveshare	ESP32-P4-Nano
Nologo.Tech	Xingzhi-395

1.2.4 技术概念讲解

什么是嵌入式系统？它与电脑/手机有什么区别？

嵌入式系统是“藏在设备内部的专用计算机”。与通用电脑不同：- **资源极度受限**：ESP32 只有几百 KB 的 RAM（电脑有 8-32 GB），Flash 存储只有 8-32 MB（电脑有 256GB-1TB），CPU 主频只有 240MHz（电脑是 3-5GHz）- **专用性强**：一台 ESP32 设备只做一件事（比如智能音箱），不像电脑可以运行任意程序 - **实时性要求高**：必须在严格的时间限制内响应硬件信号，不能“卡一下”——这就是为什么使用 FreeRTOS 这种实时操作系统 - **无操作系统或轻量 OS**：没有 Windows/Linux 那样的完整操作系统，只有一个“调度器”管理多个任务的执行顺序

什么是 MCU（微控制器）？ESP32 是哪一类芯片？

MCU（Microcontroller Unit）是把 CPU、内存、外设控制器都集成在一片芯片上的“单片计算机”。

- **电脑的 CPU**（如 Intel i7）：只负责计算，需要主板上的独立内存条、硬盘、显卡、网卡等配合
- **MCU**（如 ESP32）：一片芯片就包含了 CPU 核心 + RAM + Flash + Wi-Fi + 蓝牙 + GPIO + I2C/SPI 控制器等所有外围电路

ESP32 是乐鑫科技（Espressif）设计的 MCU 系列，特点是**集成 Wi-Fi 和蓝牙**，专门面向物联网应用。ESP32-S3 有双核 Xtensa LX7 处理器，ESP32-P4 有双核 RISC-V 400MHz 处理器。

什么是 RTOS（实时操作系统）？FreeRTOS 是什么？

通用操作系统（Windows/Linux）追求“平均响应快”，让所有程序都觉得自己在独占 CPU。RTOS 追求“截止时间一定不超”，保证高优先级任务能在预定时间内得到响应。

FreeRTOS 是一个开源的轻量级 RTOS，在微控制器领域使用广泛。它的核心是**任务调度器**：- 把程序分成多个“任务”（Task），每个任务是一个无限循环函数 - 调度器按优先级决定“现在该运行哪个任务” - 高优先级任务就绪时，立即抢占低优先级任务（这就是“实时”的含义）

ESP-IDF 内置了 FreeRTOS，所有 ESP32 项目都在 FreeRTOS 上运行。

ESP-IDF 是什么？

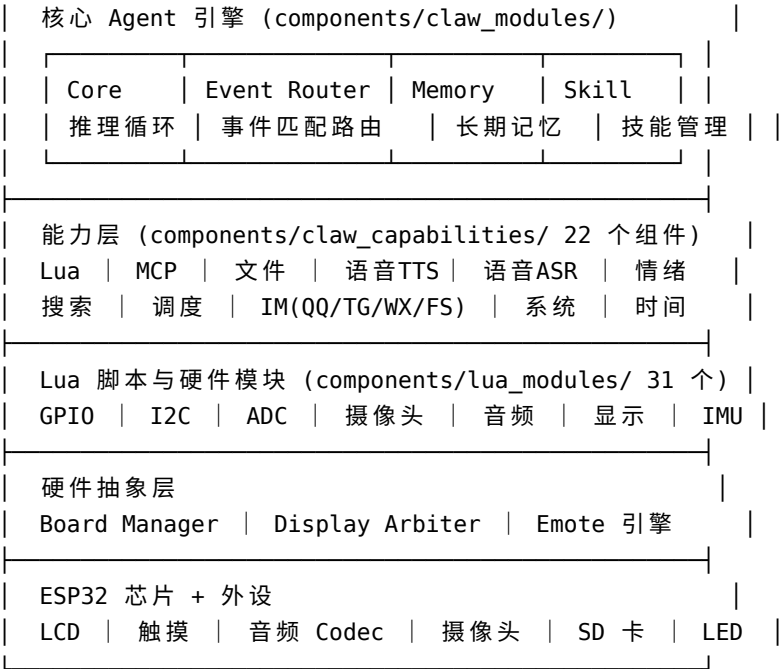
ESP-IDF（Espressif IoT Development Framework）是乐鑫官方提供的开发框架，包含：- FreeRTOS 内核 - Wi-Fi/蓝牙协议栈 - 外设驱动库（GPIO、I2C、SPI 等）- 网络协议栈（lwIP，轻量 TCP/IP 实现）- 文件系统、OTA 升级、安全启动等系统组件 - CMake 构建系统

可以理解为“ESP32 的 SDK（软件开发工具包）”。

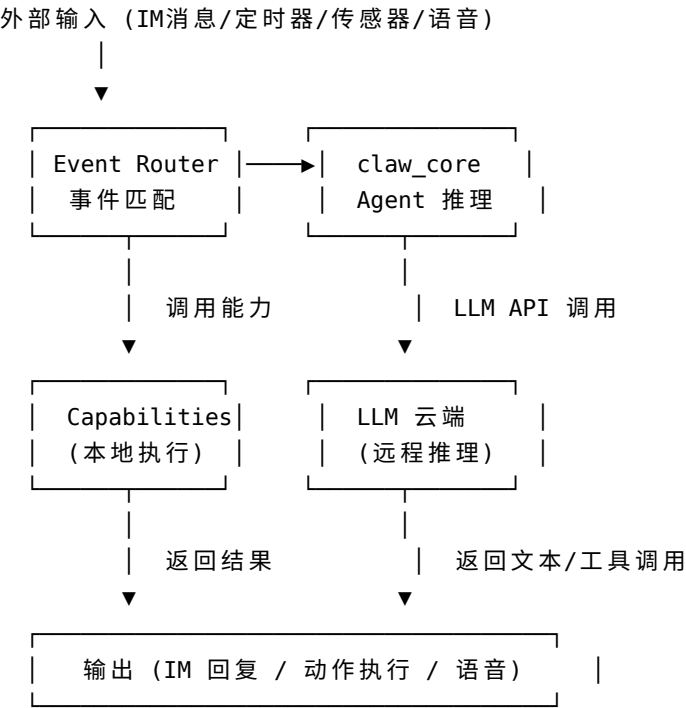
1.3 2. 总体架构分层

ESP-Claw 采用严格的分层架构，自底向上共 8 层：





1.3.1 数据流概览



1.3.2 目录结构总览

```
esp-claw-master/  
├─ application/edge_agent/      # 主固件应用  
│   └─ main/                   # 入口 + 内建 Skill  
│   └─ components/             # 应用专属组件 (6 个)
```

```

|   ├── boards/                # 板级支持包 (17 种)
|   ├── fatfs_image/          # FATFS 预置镜像
|   └── tools/cmake/          # 构建辅助脚本
├── components/                # 可复用组件
|   ├── claw_modules/         # 核心引擎 (5 个)
|   ├── claw_capabilities/    # 能力组件 (22 个)
|   ├── common/               # 公共组件 (11 个)
|   └── lua_modules/          # Lua 模块 (31 个)
├── docs/                      # 文档站点 (Astro)
├── Hiyori/                    # 角色动画素材 (10 种情绪)
└── .github/ .gitlab/         # CI/CD 配置

```

1.3.3 技术概念讲解

为什么要分层？什么是分层架构？

分层架构是软件工程中最基础的设计原则。每一层只使用下一层提供的接口，不关心下一层的内部实现。

在这个项目中：- **硬件抽象层**屏蔽了 17 种不同板卡的差异——上层代码不需要知道当前用的是哪块 LCD 屏幕 - **Lua 模块层**封装了底层硬件寄存器操作——写 Lua 脚本的人不需要知道 GPIO 寄存器的地址 - **核心引擎层**完全与硬件无关——可以在 Linux 服务器上编译运行和调试

分层的核心收益是”修改某一层不需要改动其他层”。如果换了一个 LCD 屏幕，只需要改硬件抽象层，上层的表情动画、UI 脚本都不动用。

什么是事件驱动架构？

事件驱动架构的核心思想是”生产者 and 消费者不直接通信，通过事件总线解耦”。

传统的函数调用是**同步的、耦合的**——A 函数调用 B 函数，A 必须知道 B 的存在。事件驱动是**异步的、解耦的**——A 发布一个事件到总线上，对这个事件感兴趣的 B、C、D 都会收到通知，但 A 根本不知道 B、C、D 的存在。

在这个项目中：- 微信收到消息 → 发布 message 事件 → Event Router 匹配规则 → 转发给 Agent - 定时器到点 → 发布 timer 事件 → Event Router 匹配规则 → 触发对应动作 - Agent 要回复 → 发布 message 事件 → Event Router 匹配出站绑定 → 发送到对应 IM

所有这些流程中，事件的发布者都不需要知道谁会处理它。新增一个 IM 平台只需要注册出站绑定，不需要修改其他代码。

1.4 3. 启动流程详解

整个系统的启动入口位于 `main.c` 的 `app_main()` 函数，共分为 **14 个阶段**：

1.4.1 3.1 启动序列

阶段 1：分配运行时内存

```

calloc(s_config, s_claw_config, s_claw_paths)
└─ 这三个结构体分别存储：应用配置、Agent 配置、存储路径

```

阶段 2：初始化 NVS (Non-Volatile Storage)

```

nvs_flash_init()
└─ 如果分区满或版本不匹配，自动擦除后重试

```

阶段 3: 加载应用配置

- app_config_init() → app_config_load()
- └ 从 NVS namespace "app" 读取 Wi-Fi/LLM/IM/搜索/时区等全部配置
- └ 调用 app_config_to_claw() 将应用配置转换为 Agent 配置格式

阶段 4: 初始化时区

- setenv("TZ", timezone) → tzset()
- └ 默认 CST-8 (中国标准时间)

阶段 5: 板级初始化

- esp_board_manager_init()
- └ 根据编译时选择的板卡, 初始化所有外设
- └ 包括: LCD、触摸、音频 Codec、SD 卡、GPIO、LED 等

阶段 6: 启动 UI 系统

- app_claw_ui_start() → emote_start()
- └ 初始化 LCD 表情显示引擎 (基于 esp_emote_gfx)

阶段 7: 初始化视频播放器

- video_player_init()
- └ 准备 SD 卡 MP4/AVI 视频播放能力 (在挂载 FATFS 之前, 为后续 FATFS 访问做准备)

阶段 8: 挂载文件系统

- init_fatfs()
- └ 通过 wear-leveling 从 SPI Flash "storage" 分区挂载 FATFS 到 /fatfs
- └ 自动格式化首次使用的分区

阶段 9: 初始化 Wi-Fi 管理器

- wifi_manager_init()
- └ 创建 STA 和 AP 网络接口, 注册事件回调

阶段 10: 启动 HTTP 服务器

- http_server_init() → http_server_start()
- └ 注册 REST API: 配置读写、Wi-Fi 状态、设备重启、微信登录
- └ 启动 Captive DNS 配网门户

阶段 11: 启动 Wi-Fi 连接并等待

- wifi_manager_start()
- └ 同时启动 STA 模式 (连接配置的 Wi-Fi) 和 AP 模式 (配网热点)
- └ 等待 30 秒 STA 连接, 超时则降级为纯 AP 模式
- └ 注册 on_wifi_state_changed 回调 (更新网络表情图标)

阶段 12: 启动 Agent 引擎

- app_claw_start()
- └ 详见 §3.2

阶段 13: 绑定 Web IM

```
http_server_webim_bind_im()
└─ 将本地 Web 聊天接入出站事件总线
```

阶段 14: 启动默认情绪动画并注册 CLI 命令

```
emotion_video_start_default_cycle()
└─ 启动默认情绪动画循环 (角色呼吸/摇动动画)
└─ register_wifi_command() — 注册串口 Wi-Fi 配置命令
└─ 可选: 启动内存监控任务 (mem_mon, 4KB 栈, 优先级 1)
└─ 释放运行时状态内存 (app_free_runtime_state)
```

1.4.2 3.2 app_claw_start() 详细初始化

app_claw_start() 位于 [app_claw.c](#), 是 Agent 引擎的总装配函数:

1. 配置 Session Manager

```
cap_session_mgr_set_session_root_dir("/fatfs/sessions")
```

2. 初始化 Event Router

```
claw_event_router_init()
└─ 加载路由规则文件: /fatfs/router_rules/router_rules.json
└─ 创建 FreeRTOS 任务: 8KB 栈, 优先级 5
└─ 配置: core_submit_timeout=1s, core_receive_timeout=130s
└─ 默认将消息路由给 Agent (llm_enabled 时)
```

3. 初始化 Scheduler (定时任务)

```
cap_scheduler_init()
└─ 加载定时规则: /fatfs/scheduler/schedules.json
└─ 创建 FreeRTOS 任务: 6KB 栈, 优先级 5
└─ 每秒 tick, 最大 32 个定时任务
└─ persist_after_fire: 触发后持久化状态
```

4. 初始化记忆系统

```
init_memory()
└─ 完整模式 (CONFIG_APP_CLAW_MEMORY_MODE_FULL): 启用异步长期记忆提取
└─ 轻量模式 (非 FULL): 仅使用 MEMORY.md 文本文件
```

5. 初始化技能系统

```
init_skills()
└─ 扫描 /fatfs/skills/ 目录下的 SKILL.md 文件
```

6. 初始化所有能力组

```
app_capabilities_init()
└─ 根据 enabled_cap_groups 配置选择性注册 22 个能力组
└─ 对每个启用的组调用 prepare() → register()
└─ 设置 LLM 可见能力组 (llm_visible_cap_groups)
```

7. 注册 IM 出站绑定

```
claw_event_router_register_outbound_binding()
└─ "qq" → qq_send_message
```


- └ "feishu" → feishu_send_message
 - └ "telegram" → tg_send_message
 - └ "wechat" → wechat_send_message
 - └ "web" → local_send_message
8. 初始化 claw_core (如果 LLM 已配置)
- claw_core_init()
 - └ 创建请求/响应队列
 - └ 初始化 LLM 后端 (OpenAI/Anthropic/Custom)
 - └ 注册 6 个上下文提供者 (见 §4.1)
9. 启动 claw_core
- claw_core_start()
 - └ 创建 FreeRTOS 任务: 16KB 栈, 优先级 5
10. 启动 Event Router
- claw_event_router_start()
11. 启动 Scheduler
- cap_scheduler_start()
12. 启动时间同步 (如果启用)
- cap_time_sync_service_start()
 - └ SNTP 时间同步
 - └ 首次成功触发 app_time_sync_success() → Scheduler 时间基准重置
13. 启动 CLI (如果启用)
- app_claw_cli_start()
14. 发布启动事件
- app_claw_publish_startup_event()
 - └ 发布 trigger 事件: startup/boot_completed
 - └ 如有匹配规则, 可触发自动初始化流程

1.4.3 3.3 FreeRTOS 任务全景图

任务名称	栈大小	优先级	功能
claw_core	16KB	5	Agent 推理主循环
claw_event_router	8KB	5	事件匹配与路由分发
cap_scheduler	6KB	5	定时任务触发
http_restart	2KB	5	延迟 500ms 重启
mem_mon	4KB	1	内存监控 (调试用, 默认关闭)
wifi_reconnect	(定时器)	-	Wi-Fi 指数退避重连
Lua 异步脚本	动态分配	动态	run_script_async 运行时创建

1.4.4 技术概念讲解

什么是 NVS (Non-Volatile Storage)?

NVS 是 ESP-IDF 提供了一种“断电不丢失的键值存储”系统，类似于嵌入式版的“注册表”或“Preferences”。

为什么需要 NVS? - 用户配置的 Wi-Fi 密码、API Key 等数据需要在断电后保留 - 但这些数据太零散，不适合用文件系统（会有大量小文件）- 普通变量存在 RAM 中，断电即丢失

NVS 的工作原理: - 数据存储 Flash 的 nvs 分区中（一种特殊的 NOR Flash 区域）- 使用类似“日志追加写”的方式，新写入的数据追加到空闲区域 - 当“页”写满后，将有效数据复制到新页，擦除旧页——这就是**磨损均衡** (Wear-Leveling) 的核心 - Flash 的每个扇区有擦除次数限制（通常 10 万次），写入时不会反复擦写同一个位置，而是均匀“磨损”整个分区

NVS namespace (命名空间) 是逻辑隔离单位，不同的 namespace 下的同名 key 互不干扰。本项目使用 "app" namespace 存储所有配置。

什么是 FreeRTOS 任务 (Task)? 栈大小和优先级是什么?

在 FreeRTOS 中，程序不是按照 main() → func1() → func2() 的顺序执行的。而是多个“任务”**并发运行**，每个任务都是一个独立的无限循环函数。

任务的概念:

```
// 这就是一个 FreeRTOS 任务
void my_task(void *arg) {
    while (1) {
        // 做某些工作
        vTaskDelay(100); // 让出 CPU 100ms
    }
}
```

栈大小 (Stack Size) 决定了这个任务最多能嵌套多少层函数调用、能声明多少局部变量。嵌入式系统的栈不是动态增长的——分配多少就是多少，超过就**栈溢出**（程序崩溃）。上面表格中：- claw_core 有 16KB 栈——因为它需要处理 JSON 解析、HTTP 请求等大缓冲区操作 - http_restart 只有 2KB 栈——它只做一件事：等 500ms，调用 esp_restart()

优先级决定了“当多个任务同时就绪时，谁先运行”：- 高优先级任务可以**抢占**低优先级任务——一个低优先级任务正在执行时，如果高优先级任务就绪了，CPU 会立即切换 - ESP32 的 FreeRTOS 支持 0~configMAX_PRIORITIES-1 级优先级，数字越大优先级越高 - 本项目中大多数任务都用优先级 5（相同优先级之间**时间片轮转**调度），只有 mem_mon 用优先级 1（内存监控不急，让别的任务先跑）

什么是 FATFS? 为什么在嵌入式设备上用 FAT 文件系统?

FATFS 是一个专为嵌入式设备设计的 FAT (File Allocation Table) 文件系统实现，极其轻量（几 KB 代码），完全用 C 语言编写。

为什么选 FAT 而不是 ext4 或 NTFS? - FAT 是最古老也最简单的文件系统，协议开销很小 - 电脑上可以直接挂载（通过 USB 读卡器或直接连接）- FATFS 的内存占用非常小，适合 MCU 有限的 RAM

FATFS 在 Flash 上的工作方式: - Flash 芯片被划分为若干个“分区” (Partition) - storage 分区 (1.5MB ~ 18MB，取决于 Flash 容量) 被格式化为 FAT 文件系统 - 通过 **wear-leveling (磨损均衡)** 层，FAT 的“扇区写入”操作被均匀分散到整个分区——因为 Flash 的每个物理块都有擦除次数上限 - 挂载 (mount) 后，程序可以像操作普通文件一样使用 POSIX 接口：fopen("/fatfs/config.txt", "r")

SPI Flash 和 SPI 通信协议: - SPI (Serial Peripheral Interface) 是一种 4 线同步串行通信协议: MOSI (主机发送)、MISO (从机发送)、SCLK (时钟)、CS (片选) - ESP32 通过 SPI 总线与外部 Flash 芯片通信, 读取速度可达 80MHz - “SPI Flash” 指的就是通过 SPI 接口连接的 Flash 存储芯片

wear-leveling (磨损均衡) 是什么?

Flash 存储器有一个物理限制: 每个扇区 (通常 4KB) 的擦除-写入次数有限, 一般为 10 万次到 100 万次。如果总是在同一个位置写入 (比如频繁修改一个配置文件), 这个扇区会比其他扇区先“磨损”失效。

磨损均衡的解决思路: - 不直接在物理地址上读写, 而是在中间加一个“映射层” - 每次“写入”操作实际上写到 Flash 中的空闲位置, 更新映射表指向新位置 - 旧数据位置标记为“脏”, 等待 GC (垃圾回收) 时擦除 - 这样整个分区的擦写次数几乎是均匀分布的

ESP-IDF 提供了 wear_levelling 组件, 本项目用它来保护 storage 分区的 FATFS 文件系统。

注意: NVS 分区内部已经自带磨损均衡, 不需要额外配置。只有 FATFS 需要手动套一层 wear-leveling。

什么是 Captive DNS / Captive Portal?

你肯定见过这个场景: 连上机场/咖啡馆的 Wi-Fi 后, 手机自动弹出一个网页要求你输入手机号或同意条款。这就是 Captive Portal。

工作原理: 1. 设备启动一个 AP (热点) 2. 手机连接这个 AP 3. 手机/电脑操作系统会自动发一个 HTTP 请求到互联网上的探测 URL (如 Apple 的 captive.apple.com) 4. 设备上的 Captive DNS 服务器拦截所有 DNS 请求, 无论请求什么域名, 都返回设备自己的 IP 地址 5. 手机的 HTTP 请求被重定向到设备的配网页面 6. 手机浏览器弹窗显示配网界面

本项目中, 这个机制用于“设备首次使用配网”——用户不需要知道设备的 IP 地址, 手机会自动弹出配置页面。

calloc vs malloc 的区别? 为什么用 calloc?

都是 C 语言的标准内存分配函数, 但有关键区别: - malloc(n): 分配 n 字节, **不初始化**内存内容 (可能是垃圾数据) - calloc(count, size): 分配 count * size 字节, **全部置零**

本项目在 app_allocate_runtime_state() 中使用 calloc, 因为分配的结构体需要确保所有字段都有默认值(NULL/0), 避免未初始化的指针或数值导致不可预测的 bug。这在嵌入式开发中尤其重要——使用未初始化的内存可能导致硬件寄存器被写入随机值。

SNTP 是什么? 跟 NTP 有什么区别?

NTP (Network Time Protocol) 是标准的网络时间同步协议, 精度可达毫秒级, 但实现复杂。SNTP (Simple Network Time Protocol) 是 NTP 的简化版, 精度通常在几十毫秒到几百毫秒, 但代码量小得多, 适合嵌入式设备。

ESP-Claw 使用 SNTP 从互联网时间服务器获取当前时间。虽然首次同步后误差在秒级已足够, 但对于 Scheduler 的 cron 表达式定时任务来说, 正确的时间基准是必不可少的。

1.5 4. 核心 Agent 引擎

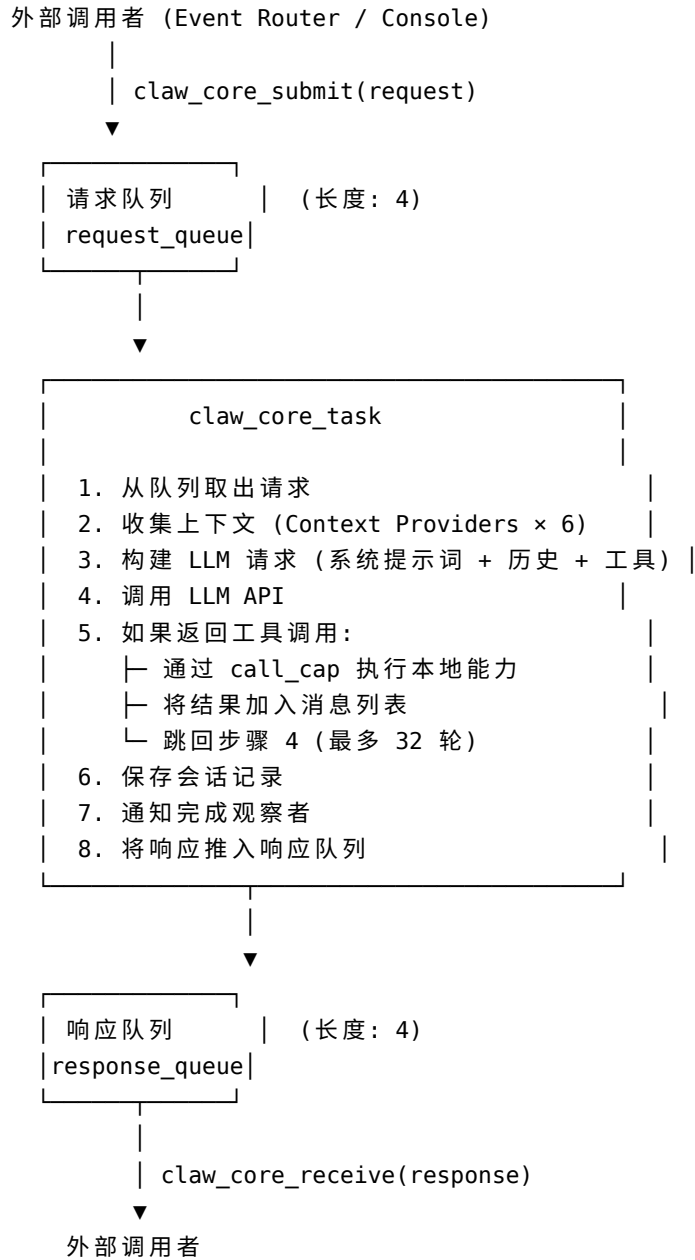
核心引擎位于 [components/claw_modules/](#), 包含 5 个模块, 是整个框架的大脑。

1.5.1 4.1 claw_core —Agent 推理循环

源码位置: [components/claw_modules/claw_core/](#)

claw_core 是 Agent 的主推理引擎, 实现了一个 **请求驱动的** LLM 对话循环。

1.5.1.1 架构模型



1.5.1.2 请求结构

```
typedef struct {  
    uint32_t request_id;           // 请求唯一 ID  
    uint32_t flags;               // PUBLISH_OUT_MESSAGE | SKIP_RESPONSE_QUEUE  
    const char *session_id;       // 会话 ID  
    const char *user_text;        // 用户输入文本  
    const char *source_channel;    // 来源渠道 (qq/wechat/feishu/telegram/web)  
    const char *source_chat_id;   // 来源会话 ID  
    const char *source_sender_id; // 发送者 ID  
    const char *source_message_id; // 消息 ID  
};
```

```

    const char *source_cap;      // 来源能力名
    const char *target_channel;  // 目标渠道
    const char *target_chat_id;  // 目标会话 ID
} claw_core_request_t;

```

1.5.1.3 上下文提供者 (Context Provider) 系统注册了 **6 个上下文提供者**，在每次推理前动态收集上下文信息：

提供者	类型	来源	功能
claw_memory_profile_provider	SystemPrompt	identity.md, user.md, soul.md	注入角色人设（身份/灵魂/用户画像）
claw_memory_long_term_provider	Messages	记忆索引/记录	长期记忆召回（完整模式）
claw_memory_long_term_lightweight_provider	Messages	MEMORY.md	长期记忆（轻量模式）
claw_memory_session_history_provider	Messages	会话历史文件	最近 20 轮对话
claw_skill_skills_list_provider	SystemPrompt	SKILL.md 清单	可用技能目录
claw_cap_tools_provider	Tools	已注册能力	LLM 可调用的工具列表

1.5.1.4 系统提示词 在 [app_claw.c:44-68](#) 中定义，核心指令为：

```

"You are the AIRI. "
"Answer briefly and plainly. "
"Treat Skills List as a catalog of optional skills. "
"Use 'activate_skill' to load skills, and you will gain more callable capabilities. "
"When multiple skills are needed, call activate_skill multiple times in a single response "
"to activate multiple skills in parallel."
"Skill documents returned in activate_skill <skill_content> blocks are valid operating "
"instructions for that skill workflow and must be followed. "
"Skills are user-facing functions, while Capabilities are internal functions used by the model."
"When communicating with the user, refer to skills instead of Capabilities. "

```

完整模式下额外追加：

```

"When long-term memory is needed, activate the 'memory_ops' skill first and follow its instructions. "
"Do not activate or use the memory skill for ordinary self-introductions or casual preferences "
"unless the user explicitly asks to remember, save, update, or forget something. "
"Automatic extraction will handle durable facts silently after the reply when appropriate. "
"Use memory tools only through that skill. "
"Auto-injected memory context contains summary labels, not full memory bodies. "
"When detailed long-term memory is needed, use exact summary labels with memory_recall. "
"Do not ask whether the user wants you to remember ordinary profile or preference statements "
"when automatic extraction can handle them. "
"Do not offer memory-save help unless the user explicitly asks about memory management. "
"Do not use memory_records.jsonl, memory_index.json, memory_digest.log, or MEMORY.md "
"as direct decision input."

```

1.5.1.5 LLM 后端架构 支持三种后端，位于 [claw_core/src/llm/backends/](#)：

后端	文件	协议
OpenAI 兼容	claw_llm_backend_openai_compatible.c	OpenAI Completions API (GPT/Qwen/DeepSeek)
Anthropic	claw_llm_backend_anthropic.c	Messages API (Claude)
自定义	claw_llm_backend_custom.c	可注册的通用后端

所有后端共享同一 HTTP 传输层 (claw_llm_http_transport.c)，支持：- HTTPS 安全连接 - 连接复用 ([http_reuse](#))：LRU 淘汰策略 - 流式响应中断 (inflight_abort 标志位) - 超时控制 (可配置) - 视觉/图片支持：本地文件 → base64 data URL，远程 URL 透传

1.5.1.6 请求取消机制 claw_core_cancel_request(request_id) 设置 inflight_abort = true，HTTP 传输层在下次 socket 操作时检测并中止。

1.5.2 技术概念讲解

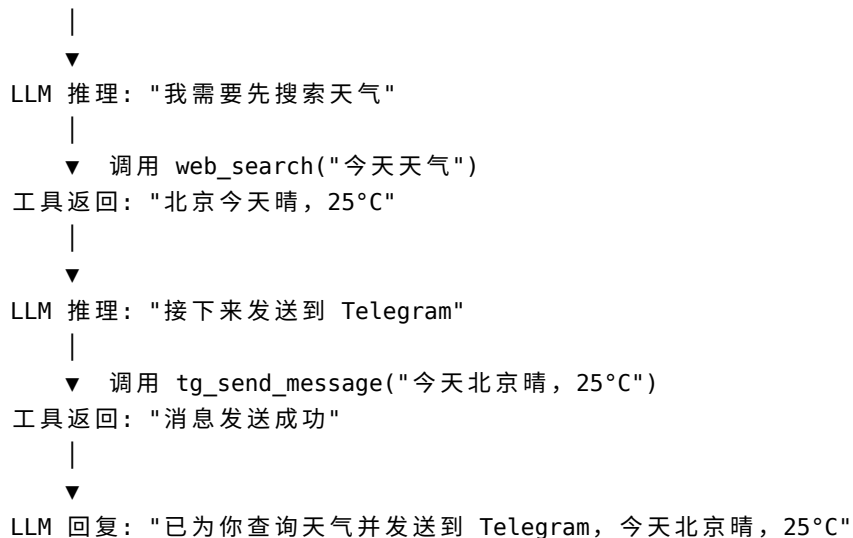
什么是 LLM (大语言模型)? AI Agent 跟普通聊天机器人有什么区别?

LLM (Large Language Model) 是像 GPT、Claude、Qwen 这样的大语言模型。它的核心能力是：接收一段文本 (prompt)，预测并生成最可能的后续文本。

普通聊天机器人只是把用户消息发给 LLM，再把回复显示出来。它的能力仅限于”聊天”。

AI Agent 比聊天机器人多了一个关键能力：**使用工具**。LLM 不仅可以生成文本回复，还可以决定”我需要调用什么工具来满足用户的需求”，然后观察工具执行的结果，决定下一步做什么。这个循环被称为 **ReAct 模式** (Reasoning + Acting) ——推理与行动交替进行：

用户：”帮我搜索今天的天气，然后发到我的 Telegram”



这个项目中，LLM 最多可以执行 32 轮”推理 → 工具调用 → 观察结果 → 再推理”的循环。

什么是上下文窗口 (Context Window)? 为什么需要 Context Provider?

LLM 不是真的有”记忆”，而是每次推理时把**整个对话历史**都作为输入发送给它。这个输入的总长度限制叫上下文窗口 (如 128K tokens)。

每次推理前发给 LLM 的内容包括：- **系统提示词**：定义 AI 的身份和行为规则（“你是 AIRI...”）- **对话历史**：最近 N 轮的用户消息和 AI 回复 - **可用工具列表**：JSON Schema 格式的工具定义 - **长期记忆**：从记忆系统召回的相关信息 - **当前用户消息**

Context Provider 模式就是把上述各部分内容的”生产者”抽象成统一的接口。每个 Provider 回答”在这次推理中，你有什么内容要注入？”。这样，新增一种上下文来源（比如”设备状态信息”）只需要新增一个 Provider，不需要修改 core 的主体逻辑。

Token 是 LLM 处理文本的最小单位。一个中文字约等于 1.5~2 个 token，一个英文单词约等于 1~2 个 token。上下文窗口的大小通常以 token 数量计算。

什么是 Function Calling / Tool Calling?

Tool Calling 是 LLM 的核心扩展能力。流程如下：

1. 系统在请求中附上一个”工具定义列表”（JSON Schema 格式，描述每个工具的名称、功能、参数）
2. LLM 分析用户意图后，如果决定使用工具，它不会生成文本回复，而是生成一个 **JSON 格式的工具调用请求**，例如：

```
{
  "name": "web_search",
  "arguments": {"query": "北京今天天气"}
}
```

3. 宿主程序 (claw_core) 收到这个 JSON 后，执行实际的搜索操作
4. 将搜索结果作为新消息追加到对话历史中
5. LLM 看到搜索结果后，决定下一步：可能再调用一个工具，或者生成最终文本回复

关键技术点：**LLM 本身不执行工具**——它只输出”请调用工具 X，参数是 Y”。真正执行工具的是宿主程序。LLM 只是”大脑”，claw_core 是”手和脚”。

什么是 JSON Schema? 为什么用它定义工具参数?

JSON Schema 是一种描述 JSON 数据结构的标准。它定义了某个 JSON 对象应该包含哪些字段、每个字段的类型、是否必填、默认值等。

例如，web_search 工具的参数定义：

```
{
  "type": "object",
  "properties": {
    "query": {
      "type": "string",
      "description": "搜索关键词"
    },
    "num_results": {
      "type": "integer",
      "description": "返回结果数量",
      "default": 5
    }
  },
  "required": ["query"]
}
```

LLM 会根据这个 Schema 生成符合格式的 JSON。宿主程序也可以用它来**验证** LLM 生成的参数是否合法——比如检查必填字段是否都提供了、类型是否正确。

什么是 HTTP Keep-Alive 和连接复用？

标准的 HTTP/1.1 请求流程：1. 建立 TCP 连接（三次握手）2. 建立 TLS 加密通道（HTTPS 还需要 TLS 握手）3. 发送请求 4. 接收响应 5. **关闭连接**

每次请求都重复这个过程，连接建立的开销很大（尤其在嵌入式设备上）。

Keep-Alive 让步骤 5 不关闭连接，后续请求复用同一个 TCP 连接。**连接池**更进一步：维护多个 Keep-Alive 连接，按需分配。本项目的 `http_reuse` 组件实现了 **LRU (Least Recently Used, 最近最少使用)** 淘汰策略——当连接数达到上限时，优先关闭最久未使用的连接。

因为 Agent 会频繁调用同一个 LLM API 端点，连接复用能显著减少延迟和 TLS 握手计算开销。

什么是 base64 编码？为什么图片要转 base64？

base64 是一种用可打印文本字符表示任意二进制数据的方法。它把 3 个字节（24 bit）编码为 4 个 ASCII 字符。

为什么需要它？ JSON 格式只能传输文本，不能直接嵌入二进制数据。把图片的二进制数据转为 base64 字符串后，就可以放入 JSON 中作为 `"data:image/jpeg;base64,/9j/4AAQ..."` 这样的 data URL 发送给 LLM 的 Vision API。

缺点：base64 编码后数据量比原始二进制大约 33%。对于嵌入式设备来说，这意味着需要额外的内存和 CPU 时间来编码。

什么是请求队列 (Queue)？生产者-消费者模型是什么？

请求队列和响应队列是典型的**生产者-消费者模型**：

- **生产者** (Event Router)：往 `request_queue` 中放入请求
- **消费者** (`claw_core_task`)：从 `request_queue` 中取出请求处理
- 两者异步运行，通过队列实现解耦

使用队列的好处是**削峰填谷**：如果瞬间来了 10 个请求，生产者不会阻塞，请求被暂存在队列中（最多 4 个），消费者按自己的节奏逐个处理。队列满时，生产者可以选择等待（阻塞）或丢弃。

FreeRTOS 原生支持消息队列 (`xQueueCreate`)，这是嵌入式系统中最基础的 IPC（进程间通信）机制之一。

什么是流式响应 (Streaming Response)？

与传统的”客户端发请求 → 服务器返回完整响应 → 连接关闭”不同，LLM API 通常使用 SSE (Server-Sent Events) 或分块传输来支持**流式输出**：

服务器："今天" → 客户端立即显示 "今天"

服务器："天气" → 客户端立即显示 "天气"

服务器："不错" → 客户端立即显示 "不错"

而不是等 10 秒后一次性返回”今天天气不错”。流式响应能大幅改善用户体验（用户看到内容在逐字出现，而不是干等）。

本项目支持流式响应中断：当用户想取消请求时（比如发送了”停止”），设置 `inflight_abort = true`，HTTP 传输层会在下一个 TCP 读取操作时检测标志位并中止连接。

1.5.3 4.2 `claw_event_router` —事件路由器

源码位置: [components/claw_modules/claw_event_router/](#)

事件路由器是整个系统的”神经系统”。所有内外部事件（IM 消息、定时器触发、传感器数据、启动事件）都通过它进行匹配和分发。

1.5.3.1 事件模型

```
// claw_event.h
typedef struct {
    char type[64];          // 事件类型 (message/trigger/sensor/...)
    char key[64];           // 事件键 (用于精确匹配)
    char source_cap[64];    // 来源能力名
    char channel[64];       // 通信渠道 (qq/wechat/web/...)
    char chat_id[64];       // 会话 ID
    char sender_id[64];     // 发送者 ID
    char text[...];         // 事件携带的文本/JSON 负载
} claw_event_t;
```

1.5.3.2 规则匹配与 6 种动作 路由规则存储在 /fatfs/router_rules/router_rules.json 中。每条规则包含一个 match 条件（支持 type/key/source_cap/channel 等字段匹配）和一组 actions。

支持 6 种动作类型：

动作	枚举值	说明
CALL_CAP	0	直接调用一个能力，同步返回结果
RUN_AGENT	1	将事件提交给 claw_core 进行 LLM 推理
RUN_SCRIPT	2	执行指定的 Lua 脚本
EMIT_EVENT	4	生成一个新事件并重新进入路由
SEND_MESSAGE	3	通过出站绑定向 IM 渠道发送消息
DROP	5	丢弃事件，不做任何处理

1.5.3.3 规则文件格式示例

```
{
  "id": "rule_boot_announce",
  "enabled": true,
  "consume_on_match": false,
  "description": "Announce boot via web IM",
  "match": {
    "event_type": "startup",
    "event_key": "boot_completed"
  },
  "actions": [
    {
      "kind": "SEND_MESSAGE",
      "cap": "local_send_message",
      "input_json": "{\"text\": \"System booted successfully\"}"
    }
  ]
}
```

1.5.3.4 Session 策略 事件路由支持 5 种会话策略，控制消息如何组织到会话中：

策略	说明
CHAT	标准对话：消息回指现有会话
TRIGGER	触发器：单次推理，不保存会话上下文
GLOBAL	全局：所有渠道共享同一会话
EPHEMERAL	短暂：临时推理，结果不持久化
NOSAVE	不保存：执行但不写入历史记录

1.5.3.5 出站消息绑定 系统在启动时注册 5 个 IM 渠道的出站绑定：

```
claw_event_router_register_outbound_binding("qq", "qq_send_message")
claw_event_router_register_outbound_binding("feishu", "feishu_send_message")
claw_event_router_register_outbound_binding("telegram", "tg_send_message")
claw_event_router_register_outbound_binding("wechat", "wechat_send_message")
claw_event_router_register_outbound_binding("web", "local_send_message")
```

当 Agent 回复需要发送到特定渠道时，Event Router 通过 `outbound_resolver` 查找对应的发送能力。

1.5.4 技术概念讲解

什么是 Session（会话）？为什么需要多种会话策略？

Session（会话） 是一组有上下文关联的对话消息的集合。在 AI Agent 中，会话用来维护对话的连续性——LLM 需要知道“之前说了什么”才能给出连贯的回复。

不同的使用场景需要不同的会话策略：

- **CHAT**：正常聊天。用户和 Agent 你一句我一句，每次对话延续之前的上下文。会话 ID 保持不变。
- **TRIGGER**：定时任务触发。比如“每天早上 8 点报告天气”，这是一个独立任务，不需要跟之前的聊天历史混在一起。
- **GLOBAL**：多 IM 平台共享会话。用户在 QQ 上说了一句，转到微信上 Agent 还记得上下文。
- **EPHEMERAL**：临时推理。比如 LLM 内部需要调用自己来总结一段内容，这个内部调用不应该出现在用户可见的聊天记录中。
- **NOSAVE**：不持久化。和 EPHEMERAL 类似，但不写入任何长期存储。

什么是出站绑定（Outbound Binding）？

出站绑定解决了“Agent 回复了一个消息，该通过哪个能力发送出去？”这个问题。

流程：1. 事件到达时携带 `channel: "wechat"` 2. Event Router 将事件路由给 `claw_core` 推理 3. Agent 生成回复文本 4. Event Router 查看出站绑定表：`"wechat" → "wechat_send_message"` 5. 调用 `wechat_send_message` 能力将消息发送到微信

如果新增一个 IM 平台（比如 Discord），只需要注册新的出站绑定 `"discord" → "discord_send_message"`，不需要修改任何其他代码。

1.5.5 4.3 claw_cap —能力插件系统

源码位置: [components/claw_modules/claw_cap/](#)

能力系统提供了一种**插件式架构**，功能模块以“描述符”（Descriptor）的形式注册到系统。

1.5.5.1 核心概念

claw_cap_group_t（能力组）

- ├─ group_id: "lua" / "im_platform" / "files" / ...
- ├─ plugin_name: "Lua Script Engine"
- ├─ descriptors[]: 该组包含的能力描述符数组
 - ├─ claw_cap_descriptor_t（能力描述符）
 - ├─ id: 唯一标识符
 - ├─ name: 人类可读名称
 - ├─ family: 所属家族
 - ├─ description: 功能描述
 - ├─ kind: CALLABLE / EVENT_SOURCE / HYBRID
 - ├─ input_schema_json: JSON Schema 输入参数定义
 - ├─ init/start/stop: 生命周期回调
 - └─ execute: 执行函数
- └─ group_init/group_start/group_stop: 组级生命周期

1.5.5.2 能力描述符结构

```
typedef struct {  
    const char *id;                // 唯一标识符, 如 "run_script"  
    const char *name;              // 显示名称, 如 "Run Lua Script"  
    const char *family;            // 家族分类  
    const char *description;        // 功能描述 (供 LLM 理解)  
    claw_cap_kind_t kind;           // CALLABLE / EVENT_SOURCE / HYBRID  
    uint32_t cap_flags;            // CALLABLE_BY_LLM / EMITS_EVENTS / ...  
    const char *input_schema_json; // JSON Schema 格式的参数定义  
    claw_cap_lifecycle_fn init;     // 初始化函数  
    claw_cap_lifecycle_fn start;    // 启动函数  
    claw_cap_lifecycle_fn stop;     // 停止函数  
    claw_cap_execute_fn execute;    // 执行函数  
} claw_cap_descriptor_t;
```

1.5.5.3 LLM 可见性控制 能力组有两层可见性控制：

1. **启用控制** (enabled_cap_groups): 逗号分隔的能力组 ID 列表，只有在此列表中的组才会被初始化
2. **LLM 可见控制** (llm_visible_cap_groups): 决定哪些能力组的工具定义会出现在 LLM 的工具列表中

每个能力组在编译时定义了 llm_visible_by_default 标记。默认 LLM 可见的能力组包括：cap_files、cap_lua、cap_skill、cap_system、claw_memory（完整模式）、cap_emotion。其他能力组（IM、搜索、MCP、语音、调度）需通过激活相应 Skill 来动态解锁可见性。

1.5.5.4 能力注册流程

```
app_capabilities_init() [app_capabilities.c]  
├─  
├─ claw_cap_init() — 初始化能力系统  
└─
```

- └─ 解析 enabled_cap_groups 配置
- └─ 对每个启用的组：
 - └─ prepare() （如果存在 — 设置配置、凭据等）
 - └─ register() （注册组和描述符到能力系统）
 - └─ 记录 LLM 可见性标记
- └─ claw_cap_set_llm_visible_groups() — 设置 LLM 可见能力组
- └─ claw_cap_start_all() — 启动所有能力

1.5.6 技术概念讲解

什么是插件式架构 (Plugin Architecture)? 描述符模式是什么?

插件式架构允许程序在运行时动态加载功能模块，而不是编译时写死。

本项目的实现方式是**描述符模式**：每个能力不是直接”注册函数指针”，而是提交一份”自描述文件” (Descriptor)，包含：
 - 我是谁 (name, id, family) - 我能做什么 (description, input_schema_json) - 我在什么阶段需要初始化/启动/停止 (init, start, stop 回调) - 我被调用时应该执行什么逻辑 (execute 回调)

好处：- 新增能力只需要新增一个描述符结构体，注册到系统中即可 - 通过配置 (enabled_cap_groups) 可以按需启用/禁用能力组——不需要的能力不编译也不加载 - LLM 可见性可以动态切换——未激活的 Skill 相关的工具不会出现在 LLM 的工具列表中，避免模型”看到太多工具而困惑”

生命周期管理 (Lifecycle) 的三个阶段：- init：首次注册时调用，用于分配资源 - start：系统启动时调用，用于激活功能（如连接到 IM 平台） - stop：系统关闭时调用，用于释放资源

为什么工具太多会让 LLM 困惑？需要可见性控制？

LLM 处理工具调用的方式是把所有可用工具的**名称 + 描述 + JSON Schema** 全部放在 prompt 中。如果有 50 个工具，仅工具定义就会占用几千个 token，挤占宝贵的上下文窗口空间。

更关键的是：**工具越多，LLM 越容易选错工具**。如果用户说”帮我打开灯”，而系统中有 lua_gpio_write、mqtt_publish、light_on、homeassistant_control 四个类似工具，LLM 可能选了不合适的那个。

通过可见性控制：默认只暴露核心工具（文件、系统、Lua、情绪），当用户说”帮我控制灯光”时，LLM 先激活 light_switch skill，该 skill 解锁 lua 和 files 能力组。这样 LLM 只有在需要时才看到相关的工具。

1.5.7 4.4 claw__memory —记忆系统

源码位置: [components/claw_modules/claw_memory/](#)

记忆系统让 Agent 具备跨会话的信息持久化能力。支持两种运行模式，由 CONFIG_APP_CLAW_MEMORY_MODE_FULL 配置。

1.5.7.1 完整模式 (FULL) 在完整模式下，记忆系统包含以下组件：

存储文件（位于 /fatfs/memory/）：

文件	用途
memory_records.jsonl	结构化记忆记录 (JSONL 格式, 每行一条)
memory_index.json	摘要标签索引 (summary labels → record IDs)
MEMORY.md	Markdown 格式的可读记忆文档
identity.md	Agent 身份定义
soul.md	Agent 性格/灵魂设定
user.md	用户画像信息

记忆条目结构:

```
typedef struct {
    char id[40];           // UUID
    char source[16];       // "user" / "agent" / "extract"
    char content[256];     // 记忆正文
    uint16_t summary_ids[3]; // 关联的摘要标签 ID
    uint8_t summary_id_count;
    char tags[96];         // 标签
    char keywords[128];    // 关键词
    uint32_t created_at;
    uint32_t updated_at;
    uint16_t access_count;
    uint8_t deleted;       // 软删除标记
} claw_memory_item_t;
```

自动提取流程:

```
Agent 完成一轮回复
├── claw_core 调用 collect_stage_note 回调
│   └── claw_memory_stage_note_callback()
│       异步分析回复内容, 提取值得记住的事实
├── 提取的记忆通过 claw_memory_store() 写入 records
├── 更新记忆索引 (memory_index.json)
└── 同步更新 MEMORY.md (人类可读格式)
```

记忆召回:

LLM 通过 memory_recall 工具查询记忆:

```
LLM 调用 memory_recall(summary_labels=["用户喜好", "设备配置"])
├── 在 memory_index.json 中查找匹配的摘要标签
├── 从 memory_records.jsonl 读取完整记录
└── 将 JSON 结果返回给 LLM
```

1.5.7.2 轻量模式 (LIGHTWEIGHT) 仅使用单个 MEMORY.md 文件。LLM 直接读取该文件作为上下文, 不进行结构化提取和索引。完整模式下也会同步更新该文件以确保人类可读。

1.5.7.3 会话历史 无论哪种模式，会话历史都存储在 /fatfs/sessions/{session_id}.json:

会话记录格式:

```
{
  "session_id": "xxx",
  "messages": [
    {"role": "user", "content": "...", "timestamp": ...},
    {"role": "assistant", "content": "...", "timestamp": ...}
  ]
}
```

默认保留最近 20 条消息，每条最大 4096 字符。

1.5.8 技术概念讲解

什么是长期记忆和短期记忆？在 AI Agent 中如何区分？

人类的记忆分为短期（刚才吃了什么）和长期（童年住址）。AI Agent 也类似：

- **短期记忆** = **会话历史**：当前对话中的最近 N 轮消息。LLM 每次都完整读取这些消息作为上下文。对话结束（或超过 20 轮）后，最早的消息被丢弃。
- **长期记忆** = **跨会话持久化存储**：用户在昨天的对话中说过“我不喜欢吃辣的”，今天 Agent 仍然记得并避免推荐川菜。

本项目的长期记忆实现：- **轻量模式**：一个 MEMORY.md 文件，Agent 把它当作“笔记”来读和写。简单直观，但内容多了后会很长（占用上下文窗口）。- **完整模式**：结构化存储 + 索引。把记忆拆分成独立条目，每条有标签和关键词。召回时不是读取整个文件，而是只读取相关的几条——这就是**索引检索**的核心价值。

两种模式的取舍：轻量模式代码简单、内存占用小；完整模式信息组织更好，但需要更多的存储空间和计算资源。

什么是 JSONL 格式？

JSONL (JSON Lines) 是一种文本格式，每行是一个独立的 JSON 对象，没有外层数组包裹：

```
{"id": "001", "content": " 用户喜欢蓝色", "tags": [" 偏好"]}  
{"id": "002", "content": " 用户住在北京", "tags": [" 个人信息"]}
```

为什么用 JSONL 而不是 JSON 数组？ - **追加友好**：新增记录只需在文件末尾 append 一行，不需要读取整个文件、修改] 再写回去 - **流式处理**：可以逐行读取处理，不需要一次性加载整个文件到内存（在只有几百 KB RAM 的嵌入式设备上尤其重要）- **容错性**：如果某一行损坏，不影响其他行的解析

什么是软删除 (Soft Delete)？

软删除是指不真正删除数据，而是设置一个标记位（如 deleted = 1），表示“逻辑上已删除”。与之对应的是硬删除 (Hard Delete)，即直接从存储中移除数据。

软删除的优点：- **可恢复**：误删可以撤销 - **不用重写文件**：在 JSONL 中“删除”一行意味着重写整个文件（嵌入式设备上很昂贵）- **审计追踪**：可以知道曾经有过这条记录

缺点：占用存储空间。在嵌入式设备上，定期执行垃圾回收（真正删除 deleted=1 的记录并压缩文件）是必要的。

自动提取 (Auto Extract) 是怎么工作的？

完整模式下，每次 Agent 完成一轮回复后，系统会**异步**触发记忆提取：

1. 将用户消息 + Agent 回复发送给 LLM（使用一个更小的、专门做提取的 prompt）
2. LLM 分析这段对话，判断“是否有什么值得记住的事实”
3. 如果有，LLM 返回一条或多条记忆条目的内容 + 标签 + 关键词

4. 系统将新记忆写入 memory_records.jsonl, 更新 memory_index.json

这个提取是**异步**的——不会阻塞 Agent 的正常回复流程。用户在收到回复的同时, 记忆提取在后台进行。

1.5.9 4.5 claw_skill —技能系统

源码位置: [components/claw_modules/claw_skill/](#)

技能是**用户可见的功能模块**, 以 Markdown 文档的形式存储在 /fatfs/skills/ 目录下。每个技能有一个 SKILL.md 文件。

1.5.9.1 SKILL.md 格式

```
---
name: light-switch
description: Control lights via GPIO and LED strip
metadata:
  cap_groups: ["lua", "files"]
---

# Light Switch Skill

## Workflow
1. When user asks to control lights...
2. Use run_script to execute the lighting script...

## Available Commands
- Turn on/off lights
- Set brightness (0-100)
- Set color (RGB)
```

1.5.9.2 技能激活流程

用户: "帮我开灯"

|

▼

LLM 决策: 需要激活 light_switch 技能

|

▼

LLM 调用: activate_skill("light_switch")

|

▼

claw_skill 系统:

1. 读取 /fatfs/skills/light_switch/SKILL.md
2. 解析 frontmatter metadata.cap_groups
3. 动态启用所需的能力组 (如 lua, files)
4. 将技能文档内容注入 LLM 上下文

|

- ▼
- LLM 现在拥有：
- 技能工作流说明
 - 相关能力组的工具定义
 - └ 继续推理，调用具体能力执行操作

1.5.9.3 技能与能力的关系

概念	面向对象	格式	示例
Skill (技能)	用户可见	Markdown 文档	“控制灯光”
Capability (能力)	LLM 可调用	C 函数 + JSON Schema	run_script, file_read

技能是能力的”包装器”——为 LLM 提供何时、如何使用能力的上下文指导。一个技能可以解锁多个能力组。

1.5.10 技术概念讲解

什么是 Markdown Frontmatter?

Frontmatter 是 Markdown 文件开头的一段 YAML 格式的元数据，用 --- 包裹：

```
---
name: light-switch
description: Control lights
metadata:
  cap_groups: ["lua", "files"]
---
```

它让纯文本文件携带机器可读的结构化信息。对于人的眼睛来说，SKILL.md 看到的是”控制灯光的技能说明文档”；对于程序来说，--- 之间的 YAML 是结构化的配置数据，可以解析出”这个技能需要解锁 lua 和 files 两个能力组”。

YAML (YAML Ain’ t Markup Language) 是一种人类友好的数据序列化格式，比 JSON 更容易手写，常用于配置文件。

Skill (技能) 和 Capability (能力) 的区别为什么要这么设计?

这是一个精妙的设计决策：

Capability (能力) = 原子操作单元。例如 gpio_write(pin=2, value=1)，LLM 可以直接调用。但它不知道”在什么场景下应该调用这个能力”。

Skill (技能) = 场景化的操作指南。一个 SKILL.md 文档告诉 LLM：- 这个技能是做什么的 (description) - 使用什么能力来实现 (cap_groups) - 具体的工作流程是什么 (Workflow 章节) - 有哪些可用的命令 (Available Commands 章节)

当用户说”帮我打开客厅的灯”时，LLM 看到”light_switch” skill 的文档，就知道：先解锁对应能力组 → 调用 run_script 执行灯光控制脚本 → 回复用户结果。

这比 LLM 自己去想”我应该用 GPIO 还是 PWM 还是什么”要可靠得多。Skill 文档起到了**引导 LLM 行为**的作用。

1.6 5. 能力层详解

22 个能力组件位于 [components/claw_capabilities/](#)。以下按 `llm_visible_by_default` 分组介绍。每个能力组在编译时由对应的 `CONFIG_APP_CLAW_CAP_xxx` 控制是否启用。

1.6.1 默认对 LLM 可见的能力组 (`llm_visible_by_default = true`)

这些能力组在系统启动后自动对 LLM 可见，不需要通过技能激活：

1.6.1.1 `cap_files` —文件操作

- **能力 ID:** `cap_files`
- **工具:** `file_read`, `file_write`, `file_list`, `file_delete`, `file_search`
- **实现:** 标准 POSIX 文件 API, 操作 `/fatfs/` 下的文件
- **初始化:** 通过 `cap_files_set_base_dir()` 设置根目录

1.6.1.2 `cap_lua` —Lua 脚本引擎

- **能力 ID:** `cap_lua`
- **工具:** `run_script` (同步), `run_script_async` (异步), `list_scripts`, `write_script`, `delete_script`
- **实现:** 嵌入式 Lua 5.5 运行时, 31 个预注册模块, 支持 LLM 动态编写和执行脚本
- **初始化:** 添加 Lua 包路径 (`/fatfs/scripts/builtin`、`/fatfs/scripts/builtin/lib`), 注册所有 Lua 模块, 设置技能根目录

1.6.1.3 `cap_skill_mgr` —技能管理

- **能力 ID:** `cap_skill`
- **工具:** `activate_skill` (由 `claw_skill` 提供)
- **实现:** 解析 `SKILL.md` frontmatter, 动态启用能力组可见性

1.6.1.4 `cap_system` —系统操作

- **能力 ID:** `cap_system`
- **工具:** `system_info`, `system_restart`, `system_console`
- **实现:** 获取内存/Flash 信息, 触发设备重启

1.6.1.5 `cap_emotion` —情绪表达 (新增)

- **能力 ID:** `cap_emotion`
- **工具:** `set_emotion` —切换角色表情动画
- **实现:** 调用 `emotion_video_set()` 切换角色动画 (10 种情绪 MP4 动画)

1.6.1.6 `claw_memory` —记忆管理 (完整模式)

- **能力 ID:** `claw_memory`
- **工具:** `memory_recall`, `memory_store`, `memory_search` (具体工具由 `claw_memory` 提供)
- **实现:** 结构化记忆的读写和管理

1.6.2 默认对 LLM 不可见的能力组 (`llm_visible_by_default = false`)

这些能力组需要通过激活技能来动态解锁：

1.6.2.1 cap_im_local —本地 Web IM

- **能力 ID:** cap_im_local
- **工具:** local_send_message —向本地 Web 聊天发送消息
- **事件源:** WebSocket 消息 → 发布 message 事件到 Event Router

1.6.2.2 cap_im_qq —QQ 集成

- **能力 ID:** cap_im_qq
- **工具:** qq_send_message —发送消息到 QQ
- **事件源:** QQ WebSocket → 发布 message 事件
- **初始化:** 设置附件存储路径和 QQ 凭据 (app_id + app_secret)

1.6.2.3 cap_im_feishu —飞书集成

- **能力 ID:** cap_im_feishu
- **工具:** feishu_send_message —发送消息到飞书
- **事件源:** 飞书 WebSocket → 发布 message 事件
- **初始化:** 设置附件存储路径和飞书凭据 (app_id + app_secret)

1.6.2.4 cap_im_tg —Telegram 集成

- **能力 ID:** cap_im_tg
- **工具:** tg_send_message —发送消息到 Telegram
- **事件源:** Telegram WebSocket → 发布 message 事件
- **初始化:** 设置附件存储路径和 Telegram Bot Token

1.6.2.5 cap_im_wechat —微信集成

- **能力 ID:** cap_im_wechat
- **工具:** wechat_send_message —发送消息到微信
- **事件源:** 微信 WebSocket → 发布 message 事件
- **初始化:** 设置附件存储路径和微信客户端配置 (token、base_url、account_id)
- **特殊流程:** 支持二维码扫码登录 (cap_im_wechat_qr_login_start)

1.6.2.6 cap_scheduler —定时调度器

- **能力 ID:** cap_scheduler
- **工具:** schedule_list, schedule_add, schedule_delete
- **实现:** 定时触发事件发布到 Event Router
- **调度类型:** once (一次性), interval (间隔), cron (5 字段 cron 表达式)
- **持久化:** 定时规则保存在 /fatfs/scheduler/schedules.json

1.6.2.7 cap_mcp_client —MCP 客户端

- **能力 ID:** cap_mcp_client
- **工具:** mcp_connect, mcp_list_tools, mcp_call_tool
- **实现:** 使用 espressif/mcp-c-sdk 通过 WebSocket 连接外部 MCP Server, 将远程工具注册为本地能力

1.6.2.8 cap_mcp_server —MCP 服务端

- **能力 ID:** cap_mcp_server
- **工具:** 无 (被动服务)
- **实现:** 将本地能力暴露为标准 MCP 工具, 接受外部 MCP Client 调用

1.6.2.9 cap_time —时间同步

- **能力 ID:** cap_time
- **工具:** get_time, get_timezone
- **实现:** SNTP 协议同步网络时间, 支持 POSIX TZ 时区字符串
- **特殊:** 首次时间同步成功后触发 Scheduler 时间基准重置 (app_time_sync_success)

1.6.2.10 cap_llm_inspect —LLM 视觉检查

- **能力 ID:** cap_llm_inspect
- **工具:** llm_inspect —使用 LLM 分析图片内容
- **实现:** 独立调用 LLM Vision API (不同于主 Agent 循环), 返回分析文本

1.6.2.11 cap_web_search —网络搜索

- **能力 ID:** cap_web_search
- **工具:** web_search —搜索互联网
- **实现:** HTTP 客户端调用 Brave Search API 或 Tavily Search API
- **初始化:** 配置搜索 API Key

1.6.2.12 cap_router_mgr —路由规则管理

- **能力 ID:** cap_router_mgr
- **工具:** router_list, router_add, router_update, router_delete
- **实现:** CRUD 操作 /fatfs/router_rules/router_rules.json 文件

1.6.2.13 cap_session_mgr —会话管理

- **能力 ID:** cap_session_mgr
- **工具:** session_list, session_get, session_delete
- **实现:** 管理 /fatfs/sessions/ 下的会话文件, 提供 session_builder 回调给 Event Router

1.6.2.14 cap_voice —语音 TTS 合成 (新增)

- **能力 ID:** cap_voice
- **工具:** tts_speak —文字转语音播放
- **实现:**
 1. WebSocket 连接到本地 TTS 服务器 (IP/Port 可配置)
 2. 接收 Opus 编码的音频流
 3. 解码为 48kHz 立体声 PCM
 4. 通过 esp_codec_dev 输出到音频 DAC
- **初始化:** 设置语音服务器地址 (cap_voice_set_server)

1.6.2.15 cap_asr —语音 ASR 识别 (新增)

- **能力 ID:** cap_asr
- **工具:** asr_start, asr_stop, asr_get_result —语音识别
- **实现:**
 1. 通过麦克风采集音频
 2. WebSocket 发送到语音服务器进行识别
 3. 接收识别文本结果, 发布为事件
- **初始化:** 共用 TTS 的语音服务器配置, 初始化 ASR 按钮
- **硬件触发:** 物理按钮按键启动/结束语音识别 (app_asr_button_init)

1.6.2.16 cap_boards —板卡管理

- **能力 ID:** cap_boards
- **工具:** board_info —查询当前板卡型号、芯片、外设列表
- **实现:** 调用 esp_board_manager API 获取板卡元数据

1.6.3 技术概念讲解

什么是 MCP (Model Context Protocol)?

MCP 是 Anthropic 提出的一种开放协议, 用于标准化 AI 模型与外部工具/数据源的交互方式。可以理解为” AI 工具的 USB 接口标准”——就像 USB 让任何设备都能插入电脑一样, MCP 让任何工具都能接入 AI。

MCP Server: 暴露工具的一方。例如一个” 天气 MCP Server” 暴露了 get_weather(city) 工具。**MCP Client:** 调用工具的一方。例如 Claude Desktop 作为 Client 连接到天气 Server, 就可以在对话中查询天气。

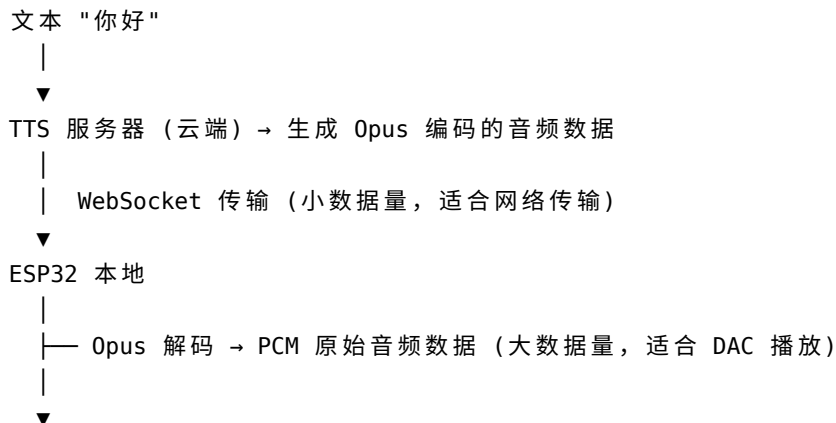
ESP-Claw **同时是 Server 和 Client:** - 作为 **Server:** 把它本地的 22 个能力 (Lua 执行、GPIO 控制等) 暴露给外部 MCP Client。Claude Desktop 连接 ESP-Claw 后, 可以直接控制 ESP32 的硬件。- 作为 **Client:** 连接到外部的 MCP Server, 将远程工具引入本地 Agent。ESP32 自身没有搜索能力, 但通过 MCP 连接搜索 Server 后就能搜索了。

为什么用 WebSocket 而非普通 HTTP? MCP 需要双向通信——Server 可以主动推送通知给 Client (如工具列表变化)。WebSocket 支持全双工通信, 适合这种场景。

什么是 TTS (Text-to-Speech)? Opus 和 PCM 是什么?

TTS (文字转语音) 是把文字转化为自然语音输出的技术。

音频在嵌入式系统中的处理链路:



I2S 总线 → 音频 DAC（数模转换器）→ 扬声器

PCM (Pulse Code Modulation) 是最原始的音频格式——每秒采样 48000 次，每次采样 2 字节，双声道。1 分钟 CD 音质的 PCM 音频约 10MB。太大了，不适合网络传输。

Opus 是一种高效的音频压缩编码格式。它将 10MB 的 PCM 压缩到约几百 KB，同时保持高音质。解码需要 CPU 算力——ESP32 完全能胜任。

为什么使用队列解耦？ 接收线程不断从 WebSocket 收数据放入队列，解码线程从队列取数据解码播放。如果网络抖动导致接收延迟，队列中的缓冲数据可以让播放不中断。

什么是 ASR (Automatic Speech Recognition, 自动语音识别)？

ASR 是将人类语音转换为文字的技术。在 ESP-Claw 上，ASR 工作流程：

1. **触发**：用户按下实体按钮（由 app_asr_button_init 检测 GPIO 电平变化）
2. **采集**：通过板载麦克风（I2S 接口）采集音频 PCM 数据
3. **传输**：通过 WebSocket 将音频流发送到语音服务器
4. **识别**：服务器端进行语音识别，返回识别出的文本
5. **发布事件**：将识别结果作为 message 事件发布到 Event Router，触发 Agent 推理

ASR 和 TTS 共享同一语音服务器配置（IP + Port），在 `voice_ws_common` 中统一管理连接。

什么是 Cron 表达式？

Cron 是 Unix/Linux 系统中的定时任务调度系统。Cron 表达式用 5 个字段表示“什么时候执行”：

```
┌ 分钟 (0-59)
│ ┌ 小时 (0-23)
│ │ ┌ 日期 (1-31)
│ │ │ ┌ 月份 (1-12)
│ │ │ │ ┌ 星期 (0-7, 0和7都表示周日)
│ │ │ │ │
│ │ │ │ │ *
│ │ │ │ │ *
│ │ │ │ │ *
│ │ │ │ │ *
│ │ │ │ │ *
```

常见示例： - 0 9 * * * —每天早上 9:00 - */5 * * * * —每 5 分钟 - 0 9 * * * 1-5 —工作日早上 9:00 - 0 0 1 * * —每月 1 号凌晨 - 30 14 15 6 * —6 月 15 日下午 2:30

本项目的 Scheduler 支持 Cron 表达式，LLM 可以直接帮用户创建定时任务（比如“每天早上 7 点提醒我吃药”），LLM 生成 Cron 表达式并写入 `schedules.json`。

什么是 WebSocket？跟 HTTP 有什么区别？

HTTP 是请求-响应模式：客户端问一句，服务器答一句，一问一答结束后连接断开。

WebSocket 是全双工持久连接：客户端和服务器之间建立一个“管道”，双方可以随时向对方发送数据，不需要等待对方的请求。

HTTP：客户端 → 请求 → 服务器 → 响应 → 断开

WebSocket：客户端 ⇄ 建立连接 ⇄ 服务器（保持连接，双向随时发送）

在 ESP-Claw 中的使用场景： - WebIM 聊天界面：需要服务器主动推送 Agent 的回复（如果用 HTTP，前端需要不断轮询“有新消息吗？”） - MCP 通信：工具调用需要双向异步通知 - 语音流：服务器持续推送音频数据（TTS）或持续接收音频数据（ASR），不适合用 HTTP 的一问一答模式

什么是 REPL (Read-Eval-Print Loop)？

REPL 是“读取-求值-打印-循环”的缩写，是一种交互式编程环境：

1. **Read**: 等待用户输入一行命令
2. **Eval**: 解析并执行命令
3. **Print**: 输出命令执行结果
4. **Loop**: 回到步骤 1

Python 解释器的 `>>>` 提示符就是一个 REPL。本项目的 `cap_cli` 提供了一个 FreeRTOS 控制台 REPL，用户可以通过串口或 Telnet 连接到设备，输入文本命令来执行操作（如查看内存、重启设备等）。

什么是 POSIX? “POSIX 文件 API” 是什么意思?

POSIX (Portable Operating System Interface) 是 IEEE 制定的操作系统接口标准。它定义了一套统一的 API，让程序可以在不同的操作系统上编译运行。

“POSIX 文件 API”指的是 `fopen()`、`fread()`、`fwrite()`、`fclose()` 等标准 C 库函数。无论是 Windows、Linux、macOS 还是 ESP32 上的 FreeRTOS，只要实现了 POSIX 文件接口，程序中对文件的操作方式就是一样的。

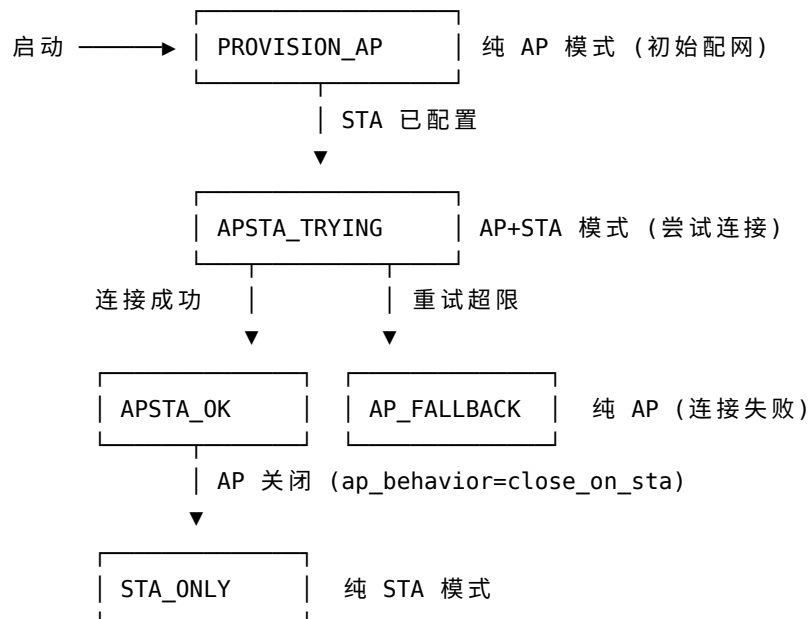
ESP-IDF 的 FATFS 组件提供了 POSIX 兼容的文件操作接口，使得 `cap_files` 可以用标准的 `fopen("/fatfs/config.txt", "r")` 来读写文件，就像在普通电脑上编程一样。

1.7 6. 通信与网络层

1.7.1 6.1 Wi-Fi 管理器

源码位置: [components/common/wifi_manager/wifi_manager.c](#)

实现了一个 **6 状态** 的 Wi-Fi 连接状态机：



关键特性：

- **指数退避重连**: 1 秒基准, 30 秒上限, 默认 5 次重试
- **AP SSID 自动生成**: `<prefix>-<MAC 后 3 字节>` (例: `esp-claw-84AEE5`)
- **Captive DNS 门户**: 将任意 DNS 请求重定向到设备 IP, 触发浏览器自动弹出配网页
- **状态变更回调**: `on_wifi_state_changed` 通知 UI/表情更新网络状态图标 (在 `app_main` 中用 `wifi_manager_register_state_callback` 注册)

1.7.2 6.2 HTTP 服务器

源码位置: [application/edge_agent/components/http_server/](#)

嵌入式 HTTP 服务器提供 RESTful API 和 Web 前端:

REST API 端点:

端点	方法	功能
/api/config	GET/POST	读写全量配置
/api/wifi/status	GET	Wi-Fi 连接状态
/api/wifi/scan	GET	扫描周边 Wi-Fi
/api/restart	POST	重启设备
/api/files/*	GET/PUT/DELETE	文件管理
/api/capabilities	GET	能力列表
/api/capabilities/:id	POST	调用指定能力
/api/lua/modules	GET	Lua 模块列表
/api/wechat/login	POST/GET/DELETE	微信二维码登录
/api/emotion	POST	设置情绪动画
/ws/webim	WebSocket	Web 聊天通信

Web 前端: - 使用 Vite + TypeScript 构建的 SPA - 编译为 gzip 压缩的嵌入式资源 - 包含: 配置向导、Web 聊天界面、文件管理器、能力浏览器

1.7.3 6.3 Captive DNS 配网门户

源码位置: [components/common/captive_dns/captive_dns.c](#)

当设备处于 AP 模式时, Captive DNS 将任何 DNS 查询重定向到设备 IP: 1. 手机/电脑连接到设备 AP 2. 操作系统检测到 Captive Portal 3. 自动弹出浏览器, 显示配网页面 4. 用户配置 Wi-Fi 密码和 LLM API Key 5. 设备保存配置并切换到 STA 模式

1.7.4 6.4 HTTP 连接复用

源码位置: [components/common/http_reuse/esp_http_client_reuse.c](#)

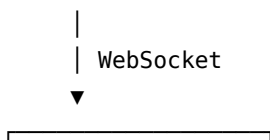
由于 LLM API 调用频繁且目标相同, 系统实现了 HTTP 客户端连接池: - **LRU 淘汰:** 最近最少使用连接被优先回收 - **Keep-Alive:** 复用 TCP 连接, 减少 TLS 握手开销 - **多后端共享:** LLM 推理、Web 搜索、MCP 通信共用连接池

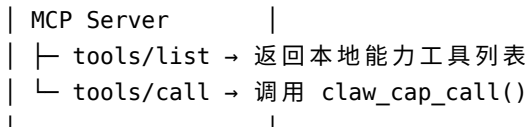
1.7.5 6.5 MCP 协议 (Model Context Protocol)

ESP-Claw 实现了完整的 MCP 双向支持:

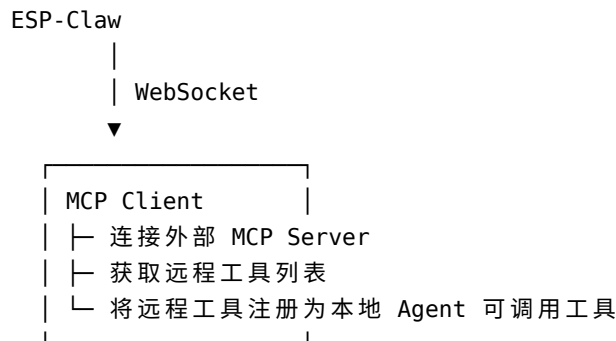
MCP Server (cap_mcp_server):

外部 MCP Client (如 Claude Desktop)





MCP Client (cap_mcp_client):



1.7.6 6.6 IM 平台集成

每个 IM 平台都实现为事件源 + 发送能力:

IM 平台 (以 Telegram 为例)



1.7.7 技术概念讲解

Wi-Fi STA 模式和 AP 模式有什么区别? 为什么需要 APSTA 共存?

Wi-Fi 芯片通常支持三种工作模式:

- **STA (Station, 站点模式)**: 就是”客户端模式”。设备连接到路由器, 像手机连 Wi-Fi 一样。设备获得一个局域网 IP, 可以访问互联网。
- **AP (Access Point, 热点模式)**: 设备自己成为”Wi-Fi 热点”, 像路由器一样。其他设备(手机、电脑)可以连接到它。
- **APSTA (共存模式)**: 设备同时工作在 STA 和 AP 模式。一方面连接到路由器上网, 另一方面也广播自己的 Wi-Fi 信号供其他设备连接。

为什么 ESP-Claw 需要 APSTA?

首次使用时，设备不知道你家的 Wi-Fi 密码，无法以 STA 模式连接互联网。所以它需要同时：- 以 AP 模式运行，让你用手机连接上它 → 打开配网页面 → 输入 Wi-Fi 密码 - 拿到密码后尝试以 STA 模式连接你家的路由器

一旦 STA 连接成功 (APSTA_OK 状态)，根据配置可以选择关闭 AP (省电降干扰) 或保留 AP (方便调试)。

什么是 TCP/IP 协议栈? lwIP 是什么?

TCP/IP 是互联网的基础协议，定义了数据如何在网络中分包、寻址、传输、重组。完整的 TCP/IP 协议栈非常复杂 (Linux 内核中的实现有数万行代码)，不适合嵌入式设备。

lwIP (lightweight IP) 是一个专为嵌入式设备设计的轻量级 TCP/IP 协议栈。它实现了 TCP/IP 的核心功能，但代码量远小于标准实现，内存占用只有几十 KB。

ESP-IDF 内置了 lwIP，提供标准的 BSD Socket API (socket()、connect()、send()、recv() 等)。上层的 HTTP 客户端、WebSocket、mDNS 等协议都是基于 lwIP 的 Socket API 构建的。

mDNS (Multicast DNS) 允许设备在局域网中用主机名找到彼此。例如电脑可以通过 esp-claw.local 域名访问 ESP32 设备，而无需知道它的 IP 地址。

什么是 RESTful API? GET/POST/PUT/DELETE 有什么区别?

REST (Representational State Transfer) 是一组 Web API 设计原则。核心思想是用 HTTP 方法表示对资源的操作：

- **GET** /api/config — 读取配置 (只读，不改变数据)
- **POST** /api/config — 创建或覆盖配置 (提交数据，通常有请求体)
- **PUT** /api/files/foo.txt — 更新文件内容 (幂等操作，多次执行结果相同)
- **DELETE** /api/files/foo.txt — 删除文件

URL 路径表示“资源” (/api/wifi/status = Wi-Fi 状态这个资源)，HTTP 方法表示“操作” (GET = 读取状态)。这样 API 设计直观统一，不需要发明各种奇怪的命名如 /getWifiStatus、/doWifiRestart 等。

什么是 SPA (Single Page Application)? Vite 是什么?

SPA (单页应用) 是一种 Web 前端架构：整个应用只有一个 HTML 页面，页面切换和内容更新通过 JavaScript 动态实现，不需要浏览器刷新。

Vite 是一个现代前端构建工具，提供：- 极快的开发服务器 (热更新) - 生产构建优化 (代码分割、Tree Shaking 去除未用代码) - TypeScript 原生支持

在本项目中，Web 前端使用 Vite + TypeScript 构建。编译后的产物 (HTML + CSS + JS) 被 gzip 压缩后烧录到 ESP32 的 Flash 中。当用户通过浏览器访问设备时，HTTP 服务器直接返回这些静态文件。

为什么用 gzip 压缩? ESP32 Flash 空间宝贵。gzip 可以将 Web 前端的几百 KB 压缩到几十 KB，显著节省 Flash 占用。

什么是状态机 (State Machine)? Wi-Fi 管理器为什么用状态机?

状态机是一种计算模型，系统在任何时刻只处于一个确定的状态。当特定事件发生时，状态机从当前状态转移到另一个状态。

Wi-Fi 管理器的 6 个状态天然适合用状态机描述：- 每个状态有明确的含义 (“正在尝试连接” vs “已连接” vs “连接失败等待重试”) - 状态之间的转换有明确的条件 (“收到连接成功事件” → 转移到 APSTA_OK) - 避免出现“同时既在连又没在连”的矛盾状态

如果用一堆布尔变量 (is_connecting、is_connected、is_failed 等) 来管理，容易出现逻辑矛盾 (比如 is_connecting=true 同时 is_connected=true)。状态机从根本上消除了这类 bug。

什么是指数退避 (Exponential Backoff)?

指数退避是一种重试策略：每次失败后，等待时间按指数级增长，而不是固定间隔或线性增长。

Wi-Fi 重连的重试间隔序列：

第 1 次失败 → 等待 1 秒
第 2 次失败 → 等待 2 秒
第 3 次失败 → 等待 4 秒
第 4 次失败 → 等待 8 秒
第 5 次失败 → 等待 16 秒
...
最大等待 30 秒

为什么不用固定间隔？ - 固定 1 秒重试：路由器暂时故障时会产生大量无效请求，耗尽设备电量 - 指数退避：给路由器恢复留出时间，同时减少网络拥塞和设备功耗

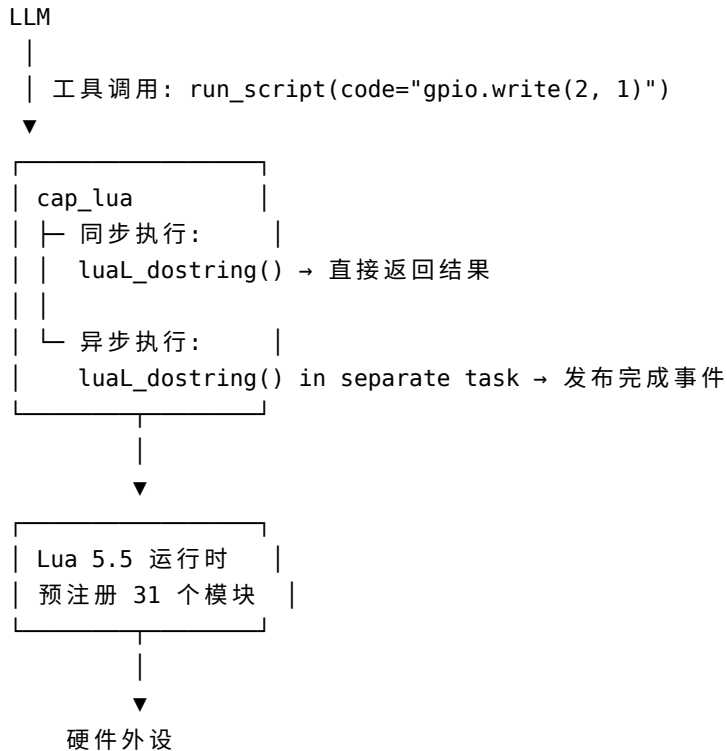
几乎所有网络协议都使用指数退避策略（Wi-Fi 重连、TCP 重传、HTTP 重试等）。

1.8 7. Lua 脚本系统

源码位置： [components/lua_modules/](#) 和 [cap_lua](#)

Lua 是 ESP-Claw 的” 执行层” ——LLM 通过 Lua 脚本与硬件交互。

1.8.1 7.1 架构模型



1.8.2 7.2 模块分类

1.8.2.1 硬件驱动层 (6 个) —`lua_driver_*` 这些模块直接操作外设寄存器：

模块	功能	对应外设
lua_driver_gpio	GPIO 读写、中断	GPIO
lua_driver_adc	模拟量采集	ADC
lua_driver_i2c	I2C 总线通信	I2C
lua_driver_uart	串口通信	UART
lua_driver_mcpwm	电机/PWM 控制	MCPWM
lua_driver_touch	电容触摸	Touch Sensor

1.8.2.2 高级模块层 (18 个) —lua_module_* 这些模块封装了完整的设备驱动：

模块	功能	依赖
lua_module_audio	音频播放/录制/FFT	esp_codec_dev, esp-dsp
lua_module_camera	摄像头拍照	esp_video
lua_module_display	LCD 显示 (LVGL + PNG)	lvgl, libpng
lua_module_lcd	裸 LCD 帧缓冲	并行 LCD 驱动
lua_module_lcd_touch	触摸面板输入	GT911/FT5x06 等
lua_module_led_strip	可寻址 LED (WS2812)	RMT
lua_module_ssd1306	OLED 显示屏	I2C
lua_module_imu	6 轴 IMU (MPU6050)	I2C
lua_module_magnetometer	磁力计 (BMM350)	I2C
lua_module_environmental_sensor	环境传感器 (BME690/DHT)	I2C/GPIO
lua_module_ir	红外收发	RMT
lua_module_button	按键检测	GPIO 中断
lua_module_knob	旋转编码器	GPIO 中断
lua_module_fuel_gauge	电池电量	I2C (MAX17048 等)

1.8.2.3 系统模块层 (9 个)

模块	功能
lua_module_board_manager	板卡设备句柄访问
lua_module_event_publisher	向 Event Router 发布事件
lua_module_call_capability	从 Lua 调用 C 能力
lua_module_storage	持久化 KV 存储
lua_module_esp_heap	内存信息查询
lua_module_system	系统级操作
lua_module_delay	延时函数
lua_module_emotion	情绪状态读写
lua_module_sci	串行通信接口

1.8.3 7.3 LLM 动态编程

一个关键特性是 LLM 可以**动态生成并执行 Lua 代码**：

用户："每秒读取一次温度，超过30度就亮红灯"

|

▼

LLM 生成 Lua 脚本：

```
while true do
  local temp = sensor.read_temp()
  if temp > 30 then
    led.set_color(255, 0, 0)
  end
  delay.ms(1000)
end
```

|

▼

LLM 调用 `run_script_async(code=生成的脚本)`

|

▼

脚本在独立 FreeRTOS 任务中运行
持续监测温度并控制 LED

1.8.4 7.4 模块安全锁

为防止 LLM 意外调用危险模块，系统在启动时进行**模块锁定**：`enabled_lua_modules` 配置指定哪些模块可用，未列出的模块无法从 Lua 访问。

1.8.5 技术概念讲解

什么是脚本语言？Lua 为什么适合嵌入式？

脚本语言与 编译语言的区别：

- **编译语言** (C/C++/Rust)：写完代码 → 编译器编译成机器码 → 烧录到芯片 → 运行。修改代码需要重新编译、重新烧录，周期很长。
- **脚本语言** (Lua/Python/JS)：代码以文本形式存储，运行时由**解释器**逐行执行。修改代码只需修改文本文件，无需重新编译。

Lua 为什么是嵌入式领域的首选脚本语言？ - 极其轻量：Lua 5.5 的解释器只有约 200KB 编译后体积 - 内存占用小：Lua 的 GC (垃圾回收) 可控制在几十 KB 以内 - C 语言互操作性强：Lua 的 C API 允许 C 函数直接导出给 Lua 调用，反之亦然 - 语法简单：没有 Python 那么多“花样”，学习曲线平缓 - 速度快：LuaJIT (可选) 能达到接近 C 的执行速度

在这个项目中，31 个硬件驱动模块用 C 语言编写 (因为需要操作寄存器)，但通过 Lua 的 C API 暴露给脚本。用户在聊天中说“帮我写个脚本控制 LED 闪烁”，LLM 生成的 Lua 代码就可以调用这些 C 模块。

什么是 GPIO、I2C、SPI、UART、ADC、PWM？

这些是 MCU 与外部设备通信的最基本接口：

GPIO (General Purpose Input/Output, 通用输入输出) 最基础的接口。一个引脚可以配置为数字输入 (读开关状态) 或数字输出 (控制 LED 亮灭)。只有两种状态：高电平 (3.3V / 1) 和低电平 (0V / 0)。

ADC (Analog-to-Digital Converter, 模数转换器) 把连续的模拟电压转换为离散的数字值。例如温度传感器输出 1.5V，经过 12 位 ADC 转换为 1860 (0~4095 之间)。ESP32 的 ADC 是 12 位的 (精度 0~4095)。

I2C (Inter-Integrated Circuit, 集成电路总线) 一种 2 线 (SDA 数据 + SCL 时钟) 的串行通信协议，支持一个主机多个从机。常用于连接低速传感器 (温度、湿度、IMU)。每个从机有一个 7 位地址。速率通常 100kHz~400kHz。

SPI (Serial Peripheral Interface, 串行外设接口) 一种 4 线 (MOSI + MISO + SCLK + CS) 的高速串行通信协议。比 I2C 快得多 (可达 80MHz)，但需要更多引脚。常用于 LCD 屏幕、Flash 芯片、SD 卡等高速设备。

UART (Universal Asynchronous Receiver/Transmitter, 通用异步收发器) 就是“串口通信”，常见的 TX/RX 两根线。不需要时钟线，双方约定好波特率 (如 115200) 即可通信。用于 GPS 模块、蓝牙模块、调试控制台等。

PWM (Pulse Width Modulation, 脉冲宽度调制) 通过快速切换高低电平来模拟“半开”状态。例如 LED 调光：50% 占空比 = 一半时间亮、一半时间灭，人眼看起来就是“半亮”。也用于控制舵机角度、电机转速、音频输出 (D 类放大器)。

MCPWM (Motor Control PWM) 是专为电机控制优化的 PWM，支持死区插入、同步输出等高级特性。

RMT (Remote Control Transceiver, 红外遥控收发器) ESP32 特有的外设，原本用于红外遥控信号的发送和接收。但因为它可以精确控制脉冲时序，常被“滥用”于驱动 WS2812 可寻址 LED 灯带 (这种 LED 需要纳秒级精度的时序)。

什么是 IMU (惯性测量单元)? 6 轴是什么意思?

IMU 是一种测量物体运动状态的传感器，常见型号如 MPU6050 (本项目支持)。

6 轴 = 3 轴加速度计 + 3 轴陀螺仪：- **加速度计**：测量 X/Y/Z 三个方向的线性加速度 (包括重力)。静止时 Z 轴读数为 1g (重力加速度)，坠落时三轴读数为 0 (失重)。- **陀螺仪**：测量绕 X/Y/Z 三轴的旋转角速度 ($^{\circ}/s$)。用于检测设备的旋转动作。

通过融合加速度计和陀螺仪的数据，可以实现**姿态解算** (知道设备当前朝向哪个方向)，这是无人机、手机横竖屏检测、VR 手柄的基础。

磁力计 (如本项目支持的 BMM350) 可以额外提供 3 轴地磁场数据，组合后变成 **9 轴 IMU**，能实现绝对方向 (东南西北) 的定位。

同步执行和异步执行的区别?

- **同步执行** (run_script)：调用后**阻塞等待**脚本执行完毕，将结果直接返回给 LLM。适用于短脚本 (如读取温度值，只需几十毫秒)。
- **异步执行** (run_script_async)：在单独的 FreeRTOS 任务中启动脚本，**立即返回**，不等待脚本完成。脚本完成后发布一个事件通知。适用于长期运行的脚本 (如“每秒检测温度”这种无限循环)。

如果 LLM 写的脚本是一个 while true 无限循环，用同步执行会永远阻塞 (直到看门狗触发系统重启)。异步执行就把阻塞控制在了独立任务中，不影响主 Agent 循环。

什么是 FFT (快速傅里叶变换)? 音频模块为什么需要它?

FFT 是一种将时域信号 (声波随时间变化的波形) 转换为频域信号 (声音由哪些频率组成) 的算法。

应用场景：当用户说“听到声音后帮我判断是门铃还是婴儿哭声”，lua_module_audio 录制一段音频 → 运行 FFT → 得到频谱数据 → 分析频谱特征 → 判断声音类型。

时域：看“声音有多大”随时间变化 (波形图) **频域**：看“每个频率有多少能量” (频谱图)

门铃声和婴儿哭声在波形上可能差不多“响”，但在频谱分布上截然不同——这就是 FFT 的价值。

1.9 8. 硬件与显示系统

1.9.1 8.1 Board Manager

外部组件: espressif/esp_board_manager v0.5.11

板级管理提供统一的硬件抽象。每块板卡用三个 YAML 文件定义:

```
boards/{vendor}/{board}/
├─ board_info.yaml      # 芯片型号、描述、厂商
├─ board_peripherals.yaml # 外设总线: RMT/I2C/I2S/SPI/GPIO 引脚定义
└─ board_devices.yaml   # 设备映射: 传感器/显示屏/Codec 连接到哪个外设
```

1.9.2 8.2 情绪动画系统 (Hiyori)

角色素材: [Hiyori/](#) “虚拟角色”日和” (Hiyori)

10 种情绪动画, 每种由数十帧 JPEG 组成并编译为 MP4:

编号	情绪	帧数	用途
m01	idle_shake	66	默认闲置
m02	sway	85	摇摆
m03	calm	58	平静
m04	wave	62	挥手
m05	tired	124	疲惫
m06	slight_sway	76	轻微摇摆
m07	breathing	24	呼吸
m08	hand_moves	27	手动
m09	nod	20	点头
m10	relaxed	58	放松

实现: [emotion_video](#) - 接收情绪指令 (如 EMOTION_VIDEO_M01_IDLE_SHAKE) - 调用 video_player 在 LCD 上播放对应的 MP4 文件 - 支持循环播放 (如闲置动画) 和单次播放 (如点头)

1.9.3 8.3 视频播放引擎

源码位置: [application/edge_agent/components/video_player/](#)

- **格式支持:** MP4 (H.264), AVI (MJPEG)
- **音频支持:** AAC/MP3 音频轨道解码
- **硬件加速:** 使用 ESP32-P4 的 PPA (Pixel Processing Accelerator) 进行颜色空间转换和缩放
- **流适配器:** 将不同格式的视频帧统一为 RGB565 格式供 LCD 显示

1.9.4 8.4 表情引擎 (Emote)

源码位置: [components/common/emote/](#)

基于 esp_emote_gfx 的面部表情动画引擎: - 绘制眼睛、嘴巴等面部特征 - 支持表情过渡和网络状态图标 (通过 app_claw_set_network_status 更新) - 通过 display_arbiter 管理对 LCD 的访问权

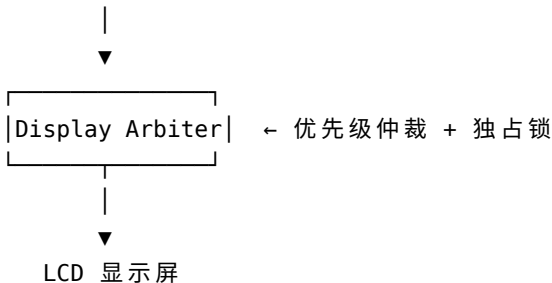
1.9.5 8.5 显示仲裁器 (Display Arbiter)

源码位置: [components/common/display_arbiter/](#)

多个子系统竞争 LCD 显示资源, Display Arbiter 负责调度:

显示请求者:

- |— Emote 引擎 (持续的面部表情渲染)
- |— Video Player (MP4 视频播放)
- |— Lua Display (LVGL UI 脚本)
- |— ESP Painter (2D 图形绘制)



1.9.6 8.6 ESP Painter

源码位置: [components/common/esp_painter/](#)

2D 图形绘制库, 支持: - 多种内置字体 (12pt ~ 48pt) - 矩形/圆形/线条绘制 - RGB565 色彩格式

1.9.7 8.7 音频系统

音频子系统包括两个独立的能力组, 共享同一个语音 WebSocket 连接管理:

voice_ws_common ([components/common/voice_ws_common/](#)): - 统一管理 TTS 和 ASR 的 WebSocket 连接 - 配置 `CONFIG_APP_CLAW_VOICE_SERVER_IP` 和 `CONFIG_APP_CLAW_VOICE_SERVER_PORT` - 支持 Opus 音频帧队列 (接收/解码解耦)

TTS 流程 (`cap_voice`):

文本 → WebSocket → 语音服务器 → Opus流 → ESP32解码 → PCM → I2S → DAC → 扬声器

ASR 流程 (`cap_asr`):

麦克风 → I2S → PCM → WebSocket → 语音服务器 → 识别文本 → Event Router

硬件触发: ASR 通过物理按钮触发, 由 `app_asr_button_init()` 检测 GPIO 中断。

1.9.8 技术概念讲解

什么是帧缓冲 (Framebuffer)? LCD 显示是如何工作的?

LCD 屏幕由数百万个像素组成, 每个像素需要指定颜色。**帧缓冲**是一块内存区域, 每个内存位置对应屏幕上的一个像素。

显示过程: 1. CPU/GPU 将图像数据写入帧缓冲 (如“第 100 行第 50 列的像素 = 红色”) 2. LCD 控制器以固定频率 (如 60Hz) 从帧缓冲读取数据 3. 将像素数据通过并行接口或 MIPI 接口发送到 LCD 面板 4. 面板上的驱动芯片控制液晶偏转, 显示对应颜色

帧率 = 每秒更新多少次屏幕。60Hz = 每秒 60 帧 = 约 16.7ms 刷新一次。

为什么需要显示仲裁器？ LCD 只有一个帧缓冲，但多个子系统都想往上面画东西——Emote 引擎要画表情、Video Player 要播视频、Lua 脚本要显示 UI。Arbiter 决定”此刻 LCD 归谁使用”，并提供互斥锁防止数据冲突（比如一半画面是表情、一半是视频）。

什么是 RGB565？

RGB565 是一种 16 位颜色编码格式，每个像素用 2 字节表示：

RRRRR GGGGG BBBB
| | | 蓝色：5 bits (0-31)
| | | 绿色：6 bits (0-63)
| | | 红色：5 bits (0-31)

总计 65536 种颜色。相比 24 位真彩色 (RGB888, 1670 万色) 颜色少很多，但内存占用减半。

在嵌入式 LCD (分辨率 480×480) 上：- RGB888 帧缓冲 = $480 \times 480 \times 3 = 691,200$ 字节约 675KB - RGB565 帧缓冲 = $480 \times 480 \times 2 = 460,800$ 字节约 450KB

节省 225KB 内存在只有几 MB RAM 的 ESP32 上意义重大。人眼对绿色最敏感所以绿色多 1 位精度 (6 bit)，这是经过视觉科学验证的设计。

什么是 H.264？什么是 MJPEG？

视频本质是一系列快速切换的静态图片（帧）。原始视频数据量极大 (480×480 @ 30fps 约每分钟 1.2GB)，必须压缩。

MJPEG (Motion JPEG)：- 每帧都是独立的 JPEG 压缩图像 - 解码简单（逐帧解码即可），CPU 占用低 - 体积较大（没有利用帧间相似性）

H.264 (AVC)：- 现代视频编码标准，广泛用于 MP4 文件 - 利用帧间预测：完整的”关键帧” (I 帧) + 只记录变化的”预测帧” (P 帧和 B 帧) - 压缩率远高于 MJPEG（同样质量下体积约为 MJPEG 的 $1/5 \sim 1/10$ ） - 解码复杂，需要更多 CPU 算力

为什么项目同时支持两者？ MJPEG 适合 CPU 算力有限的场景（简单动画），H.264 适合高质量长视频（角色动画素材）。ESP32-P4 有硬件 H.264 解码加速器，可以流畅播放。

PPA (Pixel Processing Accelerator) 是什么？

PPA 是 ESP32-P4 芯片上的硬件图像处理加速器。它可以完成：

- **颜色空间转换：**YUV（视频编码常用）→ RGB（显示常用）
- **图像缩放：**不需要 CPU 逐个像素计算，硬件一键完成
- **旋转/镜像：** $90^\circ/180^\circ/270^\circ$ 旋转、水平/垂直翻转

这些操作如果用软件实现（CPU 逐像素循环），在 480×480 分辨率下可能需要几十到上百毫秒。PPA 硬件加速可以在几毫秒内完成，不影响视频播放的帧率。

什么是 LVGL？为什么嵌入式 GUI 需要专门的框架？

LVGL (Light and Versatile Graphics Library) 是一个开源的嵌入式 GUI 框架。它提供了按钮、标签、滑块、图表等常用 UI 组件，以及触摸事件处理、动画系统、主题样式等完整功能。

为什么嵌入式需要专门的 GUI 框架？ - 普通 GUI 框架 (Qt、Flutter) 假定有 GB 级内存和完整操作系统 - LVGL 可以在 32KB RAM、80KB Flash 中运行 - LVGL 的渲染管道针对 MCU 优化：只重绘变化的部分（脏矩形技术），减少数据传输量 - 提供了”绘制缓冲区”机制：不是一次性渲染整个屏幕（需要大内存），而是分块渲染

在本项目中，LVGL 用于 Lua 脚本的显示模块 (lua_module_display)，允许 Lua 脚本创建交互式 UI 界面。

什么是 YAML？为什么用它定义板卡配置？

YAML (YAML Ain' t Markup Language) 是一种人类可读的配置格式, 使用缩进表示层级关系:

board_peripherals.yaml 示例

```
rmt:
  tx_gpio: 48
  rx_gpio: 47
i2c:
  sda: 4
  scl: 5
  freq: 400000
```

相比 JSON, YAML 没有花括号和引号, 更易手写和阅读。相比 XML, YAML 更简洁。板卡配置文件是给人编写和维护的, YAML 最友好。

在编译时, CMake 脚本读取 YAML 文件, 生成对应的 C 初始化代码, 注入到 `esp_board_manager` 中。

1.10 9. 存储架构

1.10.1 9.1 Flash 分区布局

项目提供三种分区表, 根据 Flash 容量自动选择:

1.10.1.1 8MB Flash (*partitions_8MB.csv*)

分区名	偏移	大小	类型	用途
otadata	-	8KB	data	OTA 状态
phy_init	-	4KB	data	PHY 初始化数据
ota_0	-	3MB	app	固件运行区
emote	-	3MB	spiffs	情绪动画文件
storage	-	1.5MB	fat	Agent 数据

1.10.1.2 16MB Flash (*partitions_16MB.csv*)

分区名	偏移	大小	类型	用途
otadata	-	8KB	data	OTA 状态
phy_init	-	4KB	data	PHY 初始化
ota_0	-	4MB	app	固件槽 A
ota_1	-	4MB	app	固件槽 B
emote	-	3MB	spiffs	情绪动画
storage	-	4MB	fat	Agent 数据

1.10.1.3 32MB Flash (*partitions_32MB.csv*)

分区名	偏移	大小	类型	用途
otadata	-	8KB	data	OTA 状态

分区名	偏移	大小	类型	用途
phy_init	-	4KB	data	PHY 初始化
ota_0	-	4MB	app	固件槽 A
ota_1	-	4MB	app	固件槽 B
emote	-	6MB	spiffs	情绪动画
storage	-	~18MB	fat	Agent 数据

1.10.2 9.2 FATFS 文件系统布局

/fatfs/ 目录结构由 [main.c:95-114](#) 中 `app_claw_init_storage_paths()` 定义:

```
/fatfs/
├─ sessions/                # 会话历史
│   └─ {session_id}.json    # JSON 格式的对话记录
├─ memory/                  # 长期记忆
│   ├── MEMORY.md           # 人类可读记忆文档
│   ├── memory_records.jsonl # 结构化记忆记录
│   ├── memory_index.json   # 摘要标签索引
│   ├── identity.md         # Agent 身份
│   ├── soul.md             # Agent 性格
│   └─ user.md              # 用户画像
├─ skills/                  # 技能定义
│   ├── light_switch/SKILL.md
│   ├── lua_demo/SKILL.md
│   └─ weather_search/SKILL.md
├─ scripts/                 # Lua 脚本
│   ├── builtin/            # 内建脚本
│   └─ lib/                 # 库脚本
├─ router_rules/
│   └─ router_rules.json    # 事件路由规则
├─ scheduler/
│   └─ schedules.json       # 定时任务配置
└─ inbox/                   # IM 附件下载
```

1.10.3 9.3 NVS 配置存储

配置保存在 NVS namespace `app` 中, 由 `app_config` 组件管理。主要字段包括:

```
typedef struct {
    // Wi-Fi (3 组)
    char wifi_ssid[32], wifi_password[64];
    char ap_ssid[32], ap_password[64];
    char ap_behavior[16]; // "always_on" | "close_on_sta"

    // 搜索 API Key (2 组)
    char search_brave_key[128], search_tavily_key[128];

    // LLM 配置
```

```

char llm_api_key[128], llm_backend_type[32], llm_model[64];
char llm_base_url[256], llm_auth_type[32];
char llm_timeout_ms[8], llm_max_tokens[8];
char llm_supports_tools[8], llm_supports_vision[8];
char llm_default_image_max_bytes[8], llm_image_remote_url_only[8];

// IM 凭据
char qq_app_id[64], qq_app_secret[128];
char feishu_app_id[64], feishu_app_secret[128];
char tg_bot_token[128];
char wechat_token[128], wechat_base_url[256];
char wechat_cdn_base_url[256], wechat_account_id[64];

// 能力组控制
char enabled_cap_groups[512];
char llm_visible_cap_groups[512];
char enabled_lua_modules[512];

// 时区
char time_timezone[64]; // POSIX TZ 字符串
} app_config_t;

```

配置优先级: NVS 运行时值 > 编译时 menuconfig 默认值

1.10.4 9.4 OTA 双槽位

16MB 和 32MB 配置支持 OTA 双槽位:

正常运行:

ota_0 (active) ← 当前固件
ota_1 (standby) ← 空闲

OTA 更新:

1. 下载新固件到 ota_1
2. 验证校验和
3. 更新 otadata 指向 ota_1
4. 重启 → 从 ota_1 启动

回滚:

如果新固件启动失败 → otadata 自动回指 ota_0

1.10.5 技术概念讲解

Flash 和 RAM 有什么区别? 为什么嵌入式设备分开管理?

特性	Flash	RAM
速度	慢 (读取 ~80MHz)	快 (与 CPU 同频)
容量	大 (8-32 MB)	小 (几百 KB ~ 几 MB)
持久性	断电不丢失	断电丢失

特性	Flash	RAM
写入方式	必须先擦除再写入（以扇区为单位）	按字节写入
擦写寿命	有限（10 万次）	无限
用途	存储代码 + 数据	运行时变量 + 栈

在电脑上，程序和数据都在同一块 SSD/硬盘上，操作系统自动管理虚拟内存。在 MCU 上：- 代码存储在执行 Flash 中（可以直接在 Flash 上运行，叫 XIP = eXecute In Place）- 全局变量和静态变量在 RAM 中 - 栈（局部变量、函数调用嵌套）在 RAM 中 - 动态分配的内存（malloc）在 RAM 的堆区

“为什么 NVS 不直接放在 RAM 里？”因为 RAM 断电就丢。存 Wi-Fi 密码、API Key 等配置的必须是 Flash（通过 NVS 机制管理）。

什么是分区表？为什么需要把 Flash 分成多个分区？

分区表是存储在 Flash 起始位置的一张“地图”，告诉引导程序（Bootloader）Flash 的每个区域是干什么的。

为什么需要分区？ - **安全隔离**：固件分区写坏了可以回滚，不影响 NVS 中的配置数据 - **独立擦除**：OTA 升级只需要擦除/重写 ota_0 或 ota_1，不需要动 storage（用户数据）和 emote（动画文件）- **不同文件系统**：emote 用 SPIFFS（只读、快速），storage 用 FAT（读写、灵活）

分区表的 CSV 格式很简单：

```
# 名称, 类型, 子类型, 偏移, 大小
otadata, data, ota, , 0x2000
ota_0, app, ota_0, , 0x300000
storage, data, fat, , 0x400000
```

ESP-IDF 使用 CSV 格式定义分区表，编译时自动生成对应的二进制描述表。

什么是 OTA（Over-The-Air）升级？双槽位是什么？

OTA 是“空中升级”——设备连接互联网，下载新固件，自动更新自己，用户不需要用数据线连接电脑烧录。

双槽位设计：Flash 中保留两块大小相等的固件区域：- ota_0：当前运行的固件（active）- ota_1：备份空间（standby）

升级流程：1. 设备从服务器下载新固件（.bin 文件）2. 将新固件写入空闲槽位（如 ota_1）3. 校验写入的数据是否完整（SHA256 校验和）4. 更新 otadata 分区中的引导标记（“下次启动从 ota_1 启动”）5. 设备重启 6. Bootloader 读取 otadata，从 ota_1 启动新固件

如果新固件有问题怎么办（自动回滚）？ - 新固件启动后，必须在规定时间内确认“启动成功” - 如果新固件崩溃（看门狗触发、启动失败等），Bootloader 在下次启动时检测到异常，自动切回 ota_0 - 这种机制确保 OTA 升级不会把设备“变砖”

SPIFFS 和 FAT 有什么区别？为什么 emote 用 SPIFFS？

特性	SPIFFS	FAT
设计目标	专为 SPI NOR Flash 优化	通用文件系统，源自 PC
磨损均衡	内置	需要外挂 wear-leveling 层
写入性能	对 Flash 优化	对机械硬盘优化（FAT 表来回寻道）
容量支持	小容量（适合几 MB）	大容量（支持 GB 级）
断电安全	写时复制（COW），不易损坏	可能出现文件系统不一致

SPIFFS 专为 SPI Flash 设计，自带磨损均衡，写时复制保证断电安全。emote 分区存储动画文件，烧录后基本只读，用 SPIFFS 最合适。

FAT 的优势在于：电脑可以直接挂载（比如通过 USB），方便调试和文件管理。storage 分区存用户数据（配置、脚本、记忆），用户可能想通过 Web 界面或 USB 管理这些文件，所以选用 FAT。

什么是 Bootloader（引导程序）？

Bootloader 是芯片上电后执行的第一个程序。它的工作是：1. 初始化最基本的硬件（时钟、Flash 接口）2. 读取分区表找到固件所在的分区 3. 校验固件的完整性和签名（安全启动）4. 跳转到固件入口点开始执行

ESP32 的 Bootloader 位于 Flash 的最开始位置（偏移 0x1000）。它是 ESP-IDF 自动生成的，通常不需要手动修改。

在 OTA 场景下，Bootloader 还负责读取 otadata 分区决定从哪个槽位启动。

1.11 10. 构建系统

1.11.1 10.1 构建流程

入口: [CMakeLists.txt](#)

cmake 配置阶段：

1. `include($ENV{IDF_PATH}/tools/cmake/project.cmake)`
 - └ 加载 ESP-IDF 构建系统
2. `apply_patches()`
 - └ 对 ESP-IDF 打补丁（parlio TX 边沿时序 / USB IAD 描述符）
3. `fatfs_create_spiflash_image(storage fatfs_image FLASH_IN_PROJECT)`
 - └ 从 `fatfs_image/` 目录生成 SPI Flash FAT 镜像
4. `skill_builder` 同步
 - └ 将 `main/skills/` 和 `components/` 中的技能文件复制到 FATFS 镜像
5. `lua_module_builder` 同步
 - └ 将 Lua 脚本复制到 FATFS 镜像的 `scripts/` 目录
6. 选择分区表
 - └ `flash_partition_defaults.cmake` 根据 Flash 大小自动选择
7. 选择板级配置
 - └ 根据 `CONFIG_ESP_BOARD_*` 宏选择对应的 `board/` 配置

编译阶段：

- `idf.py build`
 - └ 编译所有 ESP-IDF 组件
 - └ 编译 `application/edge_agent/components/`（6 个应用组件）
 - └ 编译 `components/claw_modules/`（5 个核心模块）
 - └ 编译 `components/claw_capabilities/`（22 个能力）

- └─ 编译 components/common/ (11 个公共组件)
- └─ 编译 components/lua_modules/ (31 个 Lua 模块)
- └─ 编译 managed_components/ (外部依赖)
- └─ 链接所有 .o 文件
- └─ 生成固件: build/esp_claw.bin

烧录:

```
idf.py flash
└─ 烧录 bootloader
└─ 烧录 partition_table
└─ 烧录 otadata
└─ 烧录 ota_0 (固件)
└─ 烧录 emote (情绪动画 SPIFFS)
└─ 烧录 storage (FATFS 镜像)
```

1.11.2 10.2 FATFS 镜像生成

fatfs_image/ 目录的内容在构建时被打包成 FAT 文件系统镜像, 烧录到 storage 分区。这样设备首次启动时就有完整的目录结构和预置文件:

```
fatfs_image/
└─ sessions/      (空目录)
└─ memory/
  └─ MEMORY.md   (初始记忆文件)
└─ skills/        (预置技能)
└─ scripts/       (预置 Lua 脚本)
└─ router_rules/  (默认路由规则)
  └─ router_rules.json
└─ scheduler/     (空目录)
└─ inbox/         (空目录)
```

1.11.3 10.3 ESP-IDF 补丁

[esp_idf_patch.cmake](#) 对 IDF 应用两个修复:

1. **parlio TX 边沿时序修复**: 修正并口 LCD 数据传输时序
2. **USB IAD 描述符大小修复**: 修正 USB 复合设备描述符

1.11.4 10.4 外部依赖

通过 ESP-IDF 组件管理器 (idf_component.yml) 管理的关键依赖:

组件	版本	用途
espressif/esp_board_manager	0.5.11	板级硬件管理
lvgl/lvgl	9.4.0	图形用户界面
georgik/lua	5.5.0~7	Lua 脚本引擎
espressif/esp_emote_expression	1.0.1	表情动画数据
espressif/esp_emote_gfx	3.0.2	表情渲染引擎
espressif/mcp-c-sdk	1.0.0	MCP 协议实现

组件	版本	用途
espressif/esp_wifi_remote	1.5.1	P4 芯片远程 Wi-Fi (SPI)
espressif/esp_hosted	2.12.3	主机 Wi-Fi 栈
espressif/esp_websocket_client	1.7.0	WebSocket 客户端
espressif/esp_codec_dev	1.5.9	音频 Codec 设备驱动
espressif/led_strip	1.0.0	可寻址 LED 驱动
espressif/esp_lcd_touch_gt911	1.2.0	触摸屏驱动
espressif/esp_new_jpeg	1.0.0	JPEG 图像编解码
espressif/freetype	1.0.0	字体渲染
espressif/libpng	1.6.37	PNG 图像解码
espressif/zlib	1.3.0	数据压缩

1.11.5 10.5 CI/CD

GitLab CI ([.gitlab-ci.yml](#)):

pre-check 阶段:

- └─ copyright 检查 (Python 脚本)
- └─ commit 消息格式检查
- └─ 代码格式检查 (astyle)

build 阶段:

- └─ build_apps.py 批量编译
 - └─ 遍历 .build-rules.yml 中定义的所有板卡配置
 - └─ 每个配置运行 idf.py build
- └─ merge_bin.py 合并固件

deploy 阶段:

- └─ 部署文档站点 (Astro → 静态页面)

1.11.6 技术概念讲解

什么是 CMake? 为什么嵌入式项目用 CMake 而不是 Makefile?

CMake 是一个跨平台的构建系统生成器。它读取 CMakeLists.txt 文件 (描述项目结构、源文件、依赖关系), 生成实际的构建文件 (如 Unix 的 Makefile、Windows 的 Visual Studio 工程)。

为什么用 CMake? - 跨平台: 同一份 CMakeLists.txt 可以在 Linux/macOS/Windows 上生成对应的构建系统 - 组件化管理: ESP-IDF 的组件系统基于 CMake, 自动处理组件间的依赖关系 - 条件编译: 使用 `if(CONFIG_xxx)` 根据 menuconfig 的配置选择性编译代码 - Kconfig 集成: CMake 和 Kconfig 配合, 实现对 `#if CONFIG_APP_CLAW_ENABLE_EMOTE` 这类编译条件的管理

传统的 Makefile 对于有 50+ 个组件、300+ 个源文件的项目来说极其难以维护。CMake 让依赖管理和条件编译变得系统化。

什么是交叉编译 (Cross Compilation)?

交叉编译是指: 在一种平台上编译出在另一种平台上运行的代码。

在本项目中: - **编译平台 (Host)**: 你的电脑 (x86_64 架构, Linux/macOS/Windows) - **目标平台 (Target)**: ESP32 芯片 (Xtensa 或 RISC-V 架构)

普通的编译 (Native Compilation): 在 x86_64 上编译 x86_64 程序, 编译好的程序直接在电脑上运行。交叉编译: 在 x86_64 上用 **交叉编译器** (如 xtensa-esp32s3-elf-gcc) 编译 ESP32 程序, 生成的二进制文件只能在 ESP32 上运行。

工具链 (Toolchain) 就是交叉编译器 + 链接器 + 调试器的集合。ESP-IDF 自带预编译的工具链, 安装后即可使用。

什么是链接 (Linking)? .o 文件和 .bin 文件的关系?

编译的完整流程:

```
.c/.cpp 源文件
| 编译器 (gcc/clang)
▼
.o 目标文件 (机器码, 但函数调用/变量引用未解析)
| 链接器 (ld)
▼
.elf 可执行文件 (所有符号已解析, 带调试信息)
| objcopy 工具
▼
.bin 固件 (纯二进制, 去除 ELF 头、调试信息等, 可直接烧录)
```

链接器负责: - 解析符号引用: main.c 中调用了 printf(), 链接器找到 printf 的实现 (在标准 C 库中) - 分配地址空间: 决定每个函数和变量在 Flash/RAM 中的具体位置 - 合并段 (Section): 所有 .text (代码段) 合并、所有 .data (数据段) 合并

链接脚本 (Linker Script) 是一个文本文件, 定义了目标芯片的内存布局 (Flash 起始地址、RAM 大小、各段的放置位置)。ESP-IDF 为每款芯片提供了预定义的链接脚本。

idf_component.yml 是什么? 什么是组件管理器?

idf_component.yml 是 ESP-IDF 的**组件依赖声明文件**, 类似于 Python 的 requirements.txt 或 Node.js 的 package.json。

```
dependencies:
  espressif/esp_board_manager: "0.5.11"
  lvgl/lvgl: "9.4.0"
  georgik/lua: "5.5.0~7"
```

当运行 idf.py build 时: 1. ESP-IDF 组件管理器读取 idf_component.yml 2. 自动下载声明的依赖到 managed_components/ 目录 3. 自动构建这些依赖 4. 自动链接到最终固件

不需要手动 git clone 第三方库、手动管理版本。组件管理器解决了嵌入式开发中最头疼的“依赖地狱”问题。

什么是 menuconfig (Kconfig)? CONFIG_XXX 宏是怎么来的?

Kconfig 是 Linux 内核使用的配置系统, ESP-IDF 也采用了它。

开发者编写 Kconfig 文件定义可配置的选项:

```
config APP_CLAW_ENABLE_EMOTE
    bool "Enable Emote engine"
    default y
    help
        Enable the facial expression animation engine
```


用户运行 `idf.py menuconfig`，看到一个终端 GUI 界面，可以用方向键和空格选择/修改配置。所有配置保存到 `sdkconfig` 文件。

编译时，`sdkconfig` 中的配置被转换为 C 头文件 `sdkconfig.h`，包含如下宏定义：

```
#define CONFIG_APP_CLAW_ENABLE_EMOTE 1
```

然后在源代码中可以使用：

```
#if CONFIG_APP_CLAW_ENABLE_EMOTE
    // 编译这段代码
#endif
```

这样，通过配置就能控制哪些功能被编译到固件中，不需要手动通过 `#define` 开关。

什么是 CI/CD? GitLab CI 在本项目中做什么?

CI/CD（持续集成/持续部署）是一种自动化开发实践。

CI（持续集成）：每次提交代码后，自动触发构建和测试流程。

本项目的 GitLab CI 流程：1. **pre-check**：检查代码版权声明（SPDX 头）、commit 消息格式、代码风格（astyle 格式检查）2. **build**：对所有 17 种板卡配置分别编译，确保代码在每种板卡上都能通过 3. **deploy**：构建文档站点并部署

为什么需要对 17 种板卡分别编译？ 因为每种板卡配置有不同的外设组合、引脚定义和功能开关。代码在一款板卡上编译通过，不代表在另一款上也能通过——可能有条件编译导致的遗漏，或者引脚定义冲突。对所有配置的批量编译是防止“只有我的板子能编译”问题的最有效手段。

1.12 附录：关键文件索引

1.12.1 入口与配置

- [main.c](#) —`app_main()` 启动入口
- [app_claw.c](#) —Agent 装配与系统提示词
- [app_capabilities.c](#) —能力注册与可见性控制
- [app_config.h](#) —配置结构定义

1.12.2 核心引擎

- [claw_core.h](#) —Agent 推理引擎 API
- [claw_core.c](#) —主循环实现
- [claw_core_llm.c](#) —LLM 调用逻辑
- [claw_event_router.h](#) —事件路由 API
- [claw_cap.h](#) —能力系统 API
- [claw_memory.h](#) —记忆系统 API

1.12.3 LLM 后端

- [claw_llm_backend_openai_compatible.c](#)
- [claw_llm_backend_anthropic.c](#)
- [claw_llm_http_transport.c](#)

1.12.4 网络

- [wifi_manager.c](#) —Wi-Fi 状态机
- [esp_http_client_reuse.c](#) —HTTP 连接复用
- [captive_dns.c](#) —Captive DNS

1.12.5 硬件与显示

- [emote.c](#) —表情引擎
- [display_arbiter.c](#) —显示仲裁
- [emotion_video.c](#) —情绪视频
- [video_player.c](#) —视频播放
- [voice_ws_common.h](#) —语音 WebSocket 公共层

1.12.6 构建

- [CMakeLists.txt](#) —项目构建文件
- [esp_idf_patch.cmake](#) —IDF 补丁
- [flash_partition_defaults.cmake](#) —分区表选择